

SOLIDWORKS API (.NET VBA C#) — Examples (19/50): Display Elevation Symbol Example VB- Expand_and_Collapse_FeatureManager_Design

SOLIDWORKS API Documentation (converted from CHM)

Display Elevation Symbol Example (VBA)

This example shows how to display an elevation symbol at the end of each ordinate dimension extension line in a part.

```
'-----
'
' Preconditions: Part document called Block.SLDPRT is
'               open and contains the selected dimensions.
'
' Postconditions: An elevation symbol and the word "Dowel" are displayed
'               at the end of each extension line of each selected dimension.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim longstatus As Long
Dim longwarnings As Long
Dim selType As Long
Dim i As Long

Sub SelectDimensions()
    boolstatus = swModelDocExt.SelectByID2("D1@Sketch1@Block.SLDPRT", "DIMENSION", -0.1000132239804, 0.1006163020026, 0.015, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("D2@Sketch1@Block.SLDPRT", "DIMENSION", -0.06157715941924, 0.1015772036167, 0.015, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("D3@Sketch1@Block.SLDPRT", "DIMENSION", -0.02362154566506, 0.1054208100728, 0.015, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("D4@Sketch1@Block.SLDPRT", "DIMENSION", 0.01481451889614, 0.1063817116868, 0.015, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("D5@Sketch1@Block.SLDPRT", "DIMENSION", 0.04940697700122, 0.1063817116868, 0.015, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("D6@Sketch1@Block.SLDPRT", "DIMENSION", 0.08592123833436, 0.1068621624938, 0.015, True, 0, Nothing, swSelectOptionDefault)
End Sub

Sub main()
    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swModelDocExt = swModelDoc.Extension
    Set swSelMgr = swModelDoc.SelectionManager

    swModelDoc.ClearSelection2 True

    Call SelectDimensions

    Dim selCount As Long
    selCount = swSelMgr.GetSelectedObjectCount

    For i = 1 To selCount

        selType = swSelMgr.GetSelectedObjectType2(i)

        If selType = swSelDIMENSIONS Then

            Set swDisplayDim = swSelMgr.GetSelectedObject5(i)

            swDisplayDim.SetText swDimensionTextAll, "Dowel"
            swDisplayDim.EndSymbol = swOrdDimEndSymbol_ Dowel
            swDisplayDim.Elevation = True

        End If

    Next i

    swModelDoc.ClearSelection2 True

End Sub
```


Display Grid Bubble Example (VBA)

This example shows how to set the text of ordinate dimensions and display a grid bubble at the end of each ordinate dimension extension line.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\2012-sm.slddrw.
'
' Postconditions:
' 1. Creates ordinate dimensions.
' 2. Selects the ordinate dimensions.
' 3. Sets the text of the ordinate dimensions to B and displays
'    a grid bubble at the end of each ordinate dimension's extension
'    line.
' 4. Examine the drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim error As Long
Dim selType As Long
Dim i As Long

Sub SelectDimensions()
boolstatus = swModelDocExt.SelectByID2("D1@Sketch3@2012-sm.SLDDRW", "DIMENSION", 8.77948412666479E-02, 0.282838220689655, 0, False, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("D2@Sketch3@2012-sm.SLDDRW", "DIMENSION", 0.100748137818372, 0.284091765517241, 0, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("D3@Sketch3@2012-sm.SLDDRW", "DIMENSION", 0.115372827473544, 0.283256068965517, 0, True, 0, Nothing, 0)
End Sub

Sub main()

Set swApp = Application.SldWorks
Set swModelDoc = swApp.ActiveDoc
Set swDrawingDoc = swModelDoc
Set swModelDocExt = swModelDoc.Extension

'Add ordinate dimensions
boolstatus = swDrawingDoc.ActivateView("Drawing View1")
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 8.69591447149237E-02, 0.257349475858578, -499.97349537912, False, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 0.101583834370096, 0.256931627582716, -499.97349537912, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 0.114954979197682, 0.256931627582716, -499.97349537912, True, 0, Nothing, 0)
error = swModelDocExt.AddOrdinateDimension(swAddOrdinateDims_e.swHorizontalOrdinate, 8.65412964390616E-02, 0.280748979310345, 0)

'Select the ordinate dimensions
Set swSelMgr = swModelDoc.SelectionManager
Call SelectDimensions

'Change dimension values to letter B and
'add grid bubble at end of extension lines
Dim selCount As Long
selCount = swSelMgr.GetSelectedObjectCount
For i = 1 To selCount
    selType = swSelMgr.GetSelectedObjectType2(i)
    If selType = swSelDIMENSIONS Then
        Set swDisplayDim = swSelMgr.GetSelectedObject6(i, -1)
        swDisplayDim.SetText swDimensionTextAll, "B"
        swDisplayDim.GridBubble = True
    End If
Next i

swModelDoc.ClearSelection2 True

End Sub
```

Display Hidden Lines Example (C#)

This example shows how to set the display mode of a model.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Press F5 five times, inspecting the graphics area each time.
//
// Postconditions: The display mode settings for the document are modified
// as specified.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace DisplayHiddenLines_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Make hidden lines visible
            swModel.ViewDisplayHiddengreyed();
            System.Diagnostics.Debugger.Break();
            // Display shaded
            swModel.ViewDisplayShaded();
            System.Diagnostics.Debugger.Break();
            // Remove hidden lines
            swModel.ViewDisplayHiddenremoved();
            System.Diagnostics.Debugger.Break();
            // Display as wireframe
            swModel.ViewDisplayWireframe();
            System.Diagnostics.Debugger.Break();
            // Toggle display of surface curvature
            swModel.ViewDisplayCurvature();
            System.Diagnostics.Debugger.Break();
            // Display the facets of the shaded model
            swModel.ViewDisplayFaceted();

        }

        public SldWorks swApp;
    }
}
```

Display Hidden Lines Example (VBA)

This example shows how to set the display mode of a model.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Press F5 five times, inspecting the graphics area each time.  
'-----  
' Postconditions: The display mode settings for the document are modified  
' as specified.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Make hidden lines visible  
    swModel.ViewDisplayHiddengreyed  
    Stop  
    ' Display shaded  
    swModel.ViewDisplayShaded  
    Stop  
    ' Remove hidden lines  
    swModel.ViewDisplayHiddenremoved  
    Stop  
    ' Display as wireframe  
    swModel.ViewDisplayWireframe  
    Stop  
    ' Toggle display of surface curvature  
    swModel.ViewDisplayCurvature  
    Stop  
    ' Display the facets of the shaded model  
    swModel.ViewDisplayFaceted  
  
End Sub
```

Display Hidden Lines Example (VB.NET)

This example shows how to set the display mode of a model.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Press F5 five times, inspecting the graphics area each time.  
'-----  
' Postconditions: The display mode settings for the document are modified  
' as specified.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()  
  
        Dim swModel As ModelDoc2  
        swModel = swApp.ActiveDoc  
  
        ' Make hidden lines visible  
        swModel.ViewDisplayHiddengreyed()  
        Stop  
        ' Display shaded  
        swModel.ViewDisplayShaded()  
        Stop  
        ' Remove hidden lines  
        swModel.ViewDisplayHiddenremoved()  
        Stop  
        ' Display as wireframe  
        swModel.ViewDisplayWireframe()  
        Stop  
        ' Toggle display of surface curvature  
        swModel.ViewDisplayCurvature()  
        Stop  
        ' Display the facets of the shaded model  
        swModel.ViewDisplayFaceted()
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Display Hidden Lines in Drawing Example (C#)

This example shows how to set the display mode of a drawing view.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Select a view.
// 3. Inspect the graphics area and press F5 six times.
//
// Postconditions: The display mode, hide/show, and suppression
// settings for the document are modified as specified.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace DisplayHiddenLinesinDrawing_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            swDraw.ViewDisplayHidden();
            System.Diagnostics.Debugger.Break();
            swDraw.ViewDisplayHiddengreyed();
            System.Diagnostics.Debugger.Break();
            swDraw.ViewDisplayWireframe();
            System.Diagnostics.Debugger.Break();
            swDraw.ViewDisplayShaded();
            System.Diagnostics.Debugger.Break();

            // Suppress view
            swDraw.SuppressView();
            System.Diagnostics.Debugger.Break();
            // Display an X where the view was suppressed
            swDraw.HideShowDrawingViews();
            System.Diagnostics.Debugger.Break();
            // Unsuppress view
            swDraw.UnsuppressView();

        }

        public SldWorks swApp;
    }
}
```

Display Hidden Lines in Drawing Example (VBA)

This example shows how to set the display mode of a drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Select a view.  
' 3. Inspect the graphics area and press F5 six times.  
'  
' Postconditions: The display mode, hide/show, and suppression  
' settings for the document are modified as specified.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
  
    ' Display mode settings  
    swDraw.ViewDisplayHidden  
    Stop  
    swDraw.ViewDisplayHiddengreyed  
    Stop  
    swDraw.ViewDisplayWireframe  
    Stop  
    swDraw.ViewDisplayShaded  
    Stop  
  
    ' Suppress view  
    swDraw.SuppressView  
    Stop  
    ' Display an X where the view was suppressed  
    swDraw.HideShowDrawingViews  
    Stop  
    ' Unsuppress view  
    swDraw.UnsuppressView  
  
End Sub
```

Display Hidden Lines in Drawing Example (VB.NET)

This example shows how to set the display mode of a drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Select a view.
' 3. Inspect the graphics area and press F5 six times.
'
' Postconditions: The display mode, hide/show, and suppression
' settings for the document are modified as specified.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc

        swModel = swApp.ActiveDoc
        swDraw = swModel

        ' Display mode settings
        swDraw.ViewDisplayHidden()
        Stop
        swDraw.ViewDisplayHiddengreyed()
        Stop
        swDraw.ViewDisplayWireframe()
        Stop
        swDraw.ViewDisplayShaded()
        Stop

        ' Suppress view
        swDraw.SuppressView()
        Stop
        ' Display an X where the view was suppressed
        swDraw.HideShowDrawingViews()
        Stop
        ' Unsuppress view
        swDraw.UnsuppressView()
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Display of Item in FeatureManager Design Tree Example (C++ COM)

This example shows how to change the visibility of features in the FeatureManager design tree. Import swconst.tlb.

```
ToggleFeatureManagerItemDisplay()
{
    VARIANT_BOOL flag;
    HRESULT hres;
    LPMODELDOC2 pModelDoc = NULL;
    hres = m_pSldWorks->get_IActiveDoc2(&pModelDoc);
    if (S_OK == hres && pModelDoc != NULL)
    {
        // Get the Selection Manager
        LPSELECTIONMGR SelectionMgr = NULL;
        hres = pModelDoc->get_ISelectionManager(&SelectionMgr);
        if (S_OK != hres || SelectionMgr == NULL )
            return;
        long numSelObjects;
        hres = SelectionMgr->GetSelectedObjectCount ( &numSelObjects );
        if (S_OK != hres || S_OK== hres && numSelObjects == 0 )
            return;

        LPUNKNOWN pSelObject;
        long index;
        long SelType;
        for (index=1; index <= numSelObjects; index++)
        {

            if(S_OK == SelectionMgr->GetSelectedObjectType2(index,&SelType))
            {
                if( SelType == swSelBODYFEATURES ||
                   SelType == swSelDATUMAXES ||
                   SelType == swSelDATUMPLANES ||
                   SelType == swSelDATUMPOINTS ||
                   SelType == swSelMATES ||
                   SelType == swSelREFCURVES ||
                   SelType == swSelREFERENCECURVES ||
                   SelType == swSelREFSURFACES ||
                   SelType == swSelSKETCHES )
                {

                    if(S_OK == SelectionMgr->IGetSelectedObject3( index,
                        &pSelObject ) && pSelObject != NULL)
                    {
                        LPFEATURE pFeature;
                        if(S_OK == pSelObject->QueryInterface(IID_IFeature,
                            (void**)&pFeature))
                        {
                            // Determine if the feature is hidden in the
                            // FeatureManager design tree
                            if(S_OK == pFeature->GetUIState(
                                swIsHiddenInFeatureMgr, &flag))
                            {
                                // If feature is hidden in the
                                // FeatureManager design tree, display it,
                                // otherwise, hide it.
                                if (flag)
                                {
                                    pFeature->SetUIState(
                                        swIsHiddenInFeatureMgr,
                                        FALSE);
                                }else{
                                    pFeature->SetUIState(
                                        swIsHiddenInFeatureMgr, TRUE);
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}
```

```
}  
}  
}  
}
```

```
// Update the Feature Manager Window by performing a Rebuild on the Doc Type  
long doctype;  
hres = pModelDoc->GetType(&doctype);
```

```
if (doctype == swDocPART )  
{  
    LPPARTDOC pPartDoc = NULL;
```

```
// Retrieve IPartDoc pointer  
hres = pModelDoc->QueryInterface(IID_IPartDoc, (LPVOID *)&pPartDoc);
```

```
ASSERT( hres == S_OK );
```

```
hres = pPartDoc->EditRebuild();
```

```
// clean up  
pPartDoc->Release(); // Release the IPartDoc pointer  
}
```

```
else if (doctype == swDocASSEMBLY)  
{  
    LPASSEMBLYDOC pAssemblyDoc = NULL;
```

```
// Retrieve IAssemblyDoc pointer  
hres = pModelDoc->QueryInterface(IID_IAssemblyDoc,  
    (LPVOID *)&pAssemblyDoc);
```

```
ASSERT( hres == S_OK );
```

```
hres = pAssemblyDoc->EditRebuild();
```

```
// clean up  
pAssemblyDoc->Release(); // Release the IAssemblyDoc pointer  
}
```

```
else if (doctype == swDocDRAWING)  
{  
    LPDRAWINGDOC pDrawingDoc = NULL;
```

```
// Retrieve IDrawingDoc pointer  
hres = pModelDoc->QueryInterface(IID_IDrawingDoc,  
    (LPVOID *)&pDrawingDoc);
```

```
ASSERT( hres == S_OK );
```

```
hres = pDrawingDoc->EditRebuild();
```

```
// clean up  
pDrawingDoc->Release(); // Release the IDrawingDoc pointer  
}
```

```
pModelDoc->Release(); // Release the IModelDoc pointer  
}  
}
```

Display Shadow Example

This example shows how to turn on shadows using `swDisplayShadowsInShadedMode`, which is a system setting.

```
iSldWorks->SetUserPreferenceToggle(swDisplayShadowsInShadedMode, TRUE);  
pInputDoc->GraphicsRedraw2();
```

Display Temporary Body Example (C#)

This example shows how to display a temporary body.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to
//    open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Selects a component for the temporary body.
// 3. Displays the temporary body.
// 4. Examine the graphics area and the Immediate
//    window.
//
// NOTE: Because the assembly is used elsewhere, do
// not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            Body2 Body = default(Body2);
            Body2 BodyCopy = default(Body2);
            bool status = false;
            Component2 Component = default(Component2);
            MathUtility MathUtility = default(MathUtility);
            MathTransform MathXform = default(MathTransform);
            SelectionMgr SelMgr = default(SelectionMgr);
            double[] Xform = new double[16];
            object vxform = null;
            int retval = 0;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            Xform[0] = 1.0;
            Xform[1] = 0.0;
            Xform[2] = 0.0;
            Xform[3] = 0.0;
            Xform[4] = 1.0;
            Xform[5] = 0.0;
            Xform[6] = 0.0;
            Xform[7] = 0.0;
            Xform[8] = 1.0;
            Xform[9] = 0.15;
            Xform[10] = 0.0;
            Xform[11] = 0.0;
            Xform[12] = 1.0;
            Xform[13] = 0.0;
            Xform[14] = 0.0;
            Xform[15] = 0.0;
            vxform = Xform;

            MathUtility = (MathUtility)swApp.GetMathUtility();
            MathXform = (MathTransform)MathUtility.CreateTransform(vxform);

            //Open assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem1.sldasm";
            Part = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Select component and create temporary body
            status = Part.Extension.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            SelMgr = (SelectionMgr)Part.SelectionManager;
            Component = (Component2)SelMgr.GetSelectedObjectsComponent3(1, 0);
            Body = (Body2)Component.GetBody();
            BodyCopy = (Body2)Body.Copy();
            BodyCopy.ApplyTransform(MathXform);

            //Display temporary body
            retval = BodyCopy.Display3(Component, 255, (int)swTempBodySelectOptions_e.swTempBodySelectable);
            Debug.Print("Temporary body displayed (0 = success)? " + retval);

            Part.ViewZoomToFit2();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Display Temporary Body Example (VBA)

This example shows how to display a temporary body.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to
'    open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Selects a component for the temporary body.
' 3. Displays the temporary body.
' 4. Examine the graphics area and the Immediate
'    window.
'
' NOTE: Because the assembly is used elsewhere, do
' not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2
    Dim Body As SldWorks.Body2
    Dim BodyCopy As SldWorks.Body2
    Dim status As Boolean
    Dim Component As SldWorks.Component2
    Dim MathUtility As SldWorks.MathUtility
    Dim MathXform As SldWorks.MathTransform
    Dim Xform(15) As Double
    Dim vXform As Variant
    Dim retVal As Long
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Xform(0) = 1#
    Xform(1) = 0#
    Xform(2) = 0#
    Xform(3) = 0#
    Xform(4) = 1#
    Xform(5) = 0#
    Xform(6) = 0#
    Xform(7) = 0#
    Xform(8) = 1#
    Xform(9) = 0.15
    Xform(10) = 0#
    Xform(11) = 0#
    Xform(12) = 1#
    Xform(13) = 0#
    Xform(14) = 0#
    Xform(15) = 0#
    vXform = Xform

    Set swApp = Application.SldWorks
    Set MathUtility = swApp.GetMathUtility
    Set MathXform = MathUtility.CreateTransform(vXform)

    'Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem1.sldasm"
    Set Part = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select component and create temporary body
    status = Part.Extension.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set Component = Part.SelectionManager.GetSelectedObjectsComponent3(1, 0)
    Set Body = Component.GetBody
    Set BodyCopy = Body.Copy
    BodyCopy.ApplyTransform MathXform

    'Display temporary body
    retVal = BodyCopy.Display3(Component, 255, swTempBodySelectable)
    Debug.Print "Temporary body displayed (0 = success)? " & retVal

    Part.ViewZoomtofit2

End Sub
```

Display Temporary Body Example (VB.NET)

This example shows how to display a temporary body.

```

-----
Preconditions:
1. Verify that the specified assembly document to
   open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified assembly document.
2. Selects a component for the temporary body.
3. Displays the temporary body.
4. Examine the graphics area and the Immediate
   window.
NOTE: Because the assembly is used elsewhere, do
not save changes.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Public Sub main()

        Dim Part As ModelDoc2
        Dim Body As Body2
        Dim BodyCopy As Body2
        Dim status As Boolean
        Dim Component As Component2
        Dim MathUtility As MathUtility
        Dim MathXform As MathTransform
        Dim Xform(15) As Double
        Dim vxform As Object
        Dim retval As Integer
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        Xform(0) = 1.0#
        Xform(1) = 0.0#
        Xform(2) = 0.0#
        Xform(3) = 0.0#
        Xform(4) = 1.0#
        Xform(5) = 0.0#
        Xform(6) = 0.0#
        Xform(7) = 0.0#
        Xform(8) = 1.0#
        Xform(9) = 0.15
        Xform(10) = 0.0#
        Xform(11) = 0.0#
        Xform(12) = 1.0#
        Xform(13) = 0.0#
        Xform(14) = 0.0#
        Xform(15) = 0.0#
        vxform = Xform

        MathUtility = swApp.GetMathUtility
        MathXform = MathUtility.CreateTransform(vxform)

        'Open assembly
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem1.sldasm"
        Part = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select component and create temporary body
        status = Part.Extension.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        Component = Part.SelectionManager.GetSelectedObjectsComponent3(1, 0)
        Body = Component.GetBody
        BodyCopy = Body.Copy
        BodyCopy.ApplyTransform(MathXform)

        'Display temporary body
        retval = BodyCopy.Display3(Component, 255, swTempBodySelectOptions_e.swTempBodySelectable)
        Debug.Print("Temporary body displayed (0 = success)? " & retval)

        Part.ViewZoomtofit2()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Display Vertices Example (VBA)

This example shows how to display vertices.

```
'-----  
'  
' Preconditions:  
' (1) Part document is open and has one or more solid bodies.  
' (2) Each solid body does not contain any circular edges.  
'  
' Postconditions: All vertices are highlighted.  
'  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.PartDoc  
Dim Body As SldWorks.Body2  
Dim Edge As SldWorks.Edge  
Dim Vertex As SldWorks.Vertex  
Dim Bodyarr As Variant  
Dim vbody As Variant  
Dim Edgearr As Variant  
Dim idx As Long  
Dim vobj As Variant  
Dim crgb As Long
```

```
Set swApp = Application.SldWorks  
Set Part = swApp.ActiveDoc
```

```
Bodyarr = Part.GetBodies2(swSolidBody, True)  
For Each vbody In Bodyarr  
    Set Body = vbody  
    Edgegearr = Body.GetEdges  
    idx = 0  
    For Each vobj In Edgegearr  
        Set Edge = vobj  
        Set Vertex = Edge.GetStartVertex  
        Select Case CInt(idx Mod 8)  
            Case 0  
                crgb = RGB(0, 0, 0)  
            Case 1  
                crgb = RGB(0, 0, 255)  
            Case 2  
                crgb = RGB(0, 255, 0)  
            Case 3  
                crgb = RGB(0, 255, 255)  
            Case 4  
                crgb = RGB(255, 0, 0)  
            Case 5  
                crgb = RGB(255, 0, 255)  
            Case 6  
                crgb = RGB(255, 255, 0)  
            Case 7  
                crgb = RGB(255, 255, 255)  
        End Select  
        idx = idx + 1  
        Vertex.Display Part, crgb, 1, True  
    Next  
Next
```

End Sub

Expand, Collapse, and Dissolve Subassembly in a BOM Table Example (C#)

This example shows how to expand, collapse, dissolve, and restore a subassembly in a BOM table.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
// 2. Select File > Make Drawing from Assembly.
// 3. Click OK.
// 4. Drag one or more views onto the drawing.
// 5. Ensure that the specified template exists.
//
// Postconditions:
// 1. Inserts an indented BOM table in the drawing.
// 2. Collapses the blade shaft subassembly. Press F5 to continue.
// 3. Expands the blade shaft subassembly. Press F5 to continue.
// 4. Dissolves the blade shaft subassembly. Press F5 to restore the blade
//    shaft subassembly.
//
// NOTE: Because this document is used by a SOLIDWORKS Design
//       online tutorial, do not save any changes when
//       closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace DissolveBOMItem_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        ModelDocExtension swModelDocExt;
        DrawingDoc swDrawing;
        View swView;
        bool boolstatus;
        BomTableAnnotation swBOMAnnotation;
        int AnchorType;
        int BomType;
        string Configuration;

        string TableTemplate;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swDrawing = (DrawingDoc)Part;
            swModelDocExt = Part.Extension;
            boolstatus = swDrawing.ActivateView("Drawing View1");
            swView = (View)swDrawing.ActiveDrawingView;

            AnchorType = (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft;
            BomType = (int)swBomType_e.swBomType_Indented;
            TableTemplate = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-standard.sldbomtbt";
            Configuration = "";
            swBOMAnnotation = swView.InsertBomTable6(false, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate, false,
            (int)swNumberingType_e.swNumberingType_Detailed, true, true, true);

            //Collapse blade shaft subassembly
            swBOMAnnotation.Collapse(1, 8);
            System.Diagnostics.Debugger.Break();

            //Expand blade shaft subassembly
            swBOMAnnotation.Expand(1, 8);
            System.Diagnostics.Debugger.Break();

            //Dissolve blade shaft subassembly
            boolstatus = swBOMAnnotation.Dissolve(8);
            System.Diagnostics.Debugger.Break();

            //Restore blade shaft subassembly
            boolstatus = swBOMAnnotation.RestoreRestructuredComponents(0);

        }

        public SldWorks swApp;
    }
}
```


Expand, Collapse, and Dissolve Subassembly in a BOM Table Example (VBA)

This example shows how to expand, collapse, dissolve, and restore a subassembly in a BOM table.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
' 2. Select File > Make Drawing from Assembly.
' 3. Click OK.
' 4. Drag one or more views onto the drawing.
' 5. Ensure that the specified template exists.
'
' Postconditions:
' 1. Inserts an indented BOM table in the drawing.
' 2. Collapses the blade shaft subassembly. Press F5 to continue.
' 3. Expands the blade shaft subassembly. Press F5 to continue.
' 4. Dissolves the blade shaft subassembly. Press F5 to restore the blade
'    shaft subassembly.
'
' NOTE: Because this document is used by a SOLIDWORKS Design
'       online tutorial, do not save any changes when
'       closing it.
'-----
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim boolstatus As Boolean
Dim swBOMAnnotation As SldWorks.BomTableAnnotation
Dim AnchorType As Long
Dim BomType As Long
Dim Configuration As String
Dim TableTemplate As String
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set swDrawing = Part
    Set swModelDocExt = Part.Extension

    boolstatus = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView
    AnchorType = swBOMConfigurationAnchor_TopLeft
    BomType = swBomType_Indented
    TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    Configuration = ""
    Set swBOMAnnotation = swView.InsertBomTable6(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate, False,
    swNumberingType_Detailed, True, True, True)

    'Collapse blade shaft subassembly
    swBOMAnnotation.Collapse 1, 8
    Stop

    'Expand blade shaft subassembly
    swBOMAnnotation.Expand 1, 8
    Stop

    'Dissolve blade shaft subassembly
    boolstatus = swBOMAnnotation.Dissolve(8)
    Stop

    'Restore blade shaft subassembly
    boolstatus = swBOMAnnotation.RestoreRestructuredComponents(0)
End Sub

```

Expand, Collapse, and Dissolve Subassembly in a BOM Table Example (VB.NET)

This example shows how to expand, collapse, dissolve, and restore a subassembly in a BOM table.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
' 2. Select File > Make Drawing from Assembly.
' 3. Click OK.
' 4. Drag one or more views onto the drawing.
' 5. Ensure that the specified template exists.
'
' Postconditions:
' 1. Inserts an indented BOM table in the drawing.
' 2. Collapses the blade shaft subassembly. Press F5 to continue.
' 3. Expands the blade shaft subassembly. Press F5 to continue.
' 4. Dissolves the blade shaft subassembly. Press F5 to restore the blade
'    shaft subassembly.
'
' NOTE: Because this document is used by a SOLIDWORKS Design
'       online tutorial, do not save any changes when
'       closing it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

Partial Class SolidWorksMacro

```

    Dim Part As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swDrawing As DrawingDoc
    Dim swView As View
    Dim boolstatus As Boolean
    Dim swBOMAnnotation As BomTableAnnotation
    Dim AnchorType As Integer
    Dim BomType As Integer
    Dim Configuration As String
    Dim TableTemplate As String

    Sub main()

        Part = swApp.ActiveDoc
        swDrawing = Part
        swModelDocExt = Part.Extension
        boolstatus = swDrawing.ActivateView("Drawing View1")
        swView = swDrawing.ActiveDrawingView

        AnchorType = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft
        BomType = swBomType_e.swBomType_Indented
        TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
        Configuration = ""
        swBOMAnnotation = swView.InsertBomTable6(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate, False,
        swNumberingType_e.swNumberingType_Detailed, True, True, True)

        'Collapse blade shaft subassembly
        swBOMAnnotation.Collapse(1, 8)
        Stop

        'Expand blade shaft subassembly
        swBOMAnnotation.Expand(1, 8)
        Stop

        'Dissolve blade shaft subassembly
        boolstatus = swBOMAnnotation.Dissolve(8)
        Stop

        'Restore blade shaft subassembly
        boolstatus = swBOMAnnotation.RestoreRestructuredComponents(0)

    End Sub

    Public swApp As SldWorks
End Class

```

Divide Sketch Segment Example (C#)

This example shows how to divide a sketch segment into equally spaced sketch points.

```
//-----
// Preconditions: Ensure that the specified template exists.
//
// Postconditions:
// 1. Creates Sketch1.
// 2. Divides the sketch's line into 99 equally spaced sketch points.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace DivideSketchSegment_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SketchSegment skSegment;
        SketchSegment SkSeg;
        bool boolstatus;
        int longstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prt", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.InsertSketch(true);
            skSegment = Part.SketchManager.CreateLine(-0.067186, 0.00851, 0.0, 0.07118, 0.081524, 0.0);
            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);

            Part = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0.0058451006543539, 0.017433416397686,
            -0.0149283857579217, false, 0, null, 0);

            SkSeg = (SketchSegment)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);
            boolstatus = SkSeg.EqualSegment((int)swSketchSegmentType_e.swSketchSegmentType_sketchpoints, 99);

            Part.ForceRebuild3(true);

            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Part.EditSketch();
            Part.SketchManager.InsertSketch(true);
        }

        public SldWorks swApp;
    }
}
```

Divide Sketch Segment Example (VBA)

This example shows how to divide a sketch segment into equally spaced sketch points.

```
'-----
' Preconditions: Ensure that the specified template exists.
```

```
' Postconditions:
' 1. Creates Sketch1.
' 2. Divides the sketch's line into 99 equally spaced sketch points.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim skSegment As SldWorks.SketchSegment
Dim SkSeg As SketchSegment
Dim boolstatus As Boolean
Dim longstatus As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set Part = swApp.ActiveDoc

    Part.SketchManager.InsertSketch True
    Set skSegment = Part.SketchManager.CreateLine(-0.067186, 0.00851, 0#, 0.07118, 0.081524, 0#)
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 5.8451006543539E-03, 0.017433416397686, -1.49283857579217E-02, False, 0, Nothing, 0)

    Set SkSeg = Part.SelectionManager.GetSelectedObject6(1, -1)
    boolstatus = SkSeg.EqualSegment(swSketchSegmentType_e.swSketchSegmentType_sketchpoints, 99)
    Part.ForceRebuild3 True

    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditSketch
    Part.SketchManager.InsertSketch True

End Sub
```

Divide Sketch Segment Example (VB.NET)

This example shows how to divide a sketch segment into equally spaced sketch points.

```
'-----
' Preconditions: Ensure that the specified template exists.
'
' Postconditions:
' 1. Creates Sketch1.
' 2. Divides the sketch's line into 99 equally spaced sketch points.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim skSegment As SketchSegment
    Dim SkSeg As SketchSegment
    Dim boolstatus As Boolean
    Dim longstatus As Integer
```

```
    Sub main()
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        Part = swApp.ActiveDoc
```

```
        Part.SketchManager.InsertSketch(True)
        skSegment = Part.SketchManager.CreateLine(-0.067186, 0.00851, 0.0#, 0.07118, 0.081524, 0.0#)
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Part = swApp.ActiveDoc
        boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0.0058451006543539, 0.017433416397686,
        -0.0149283857579217, False, 0, Nothing, 0)
```

```
        SkSeg = Part.SelectionManager.GetSelectedObject6(1, -1)
        boolstatus = SkSeg.EqualSegment(swSketchSegmentType_e.swSketchSegmentType_sketchpoints, 99)
```

```
        Part.ForceRebuild3(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        Part.EditSketch()
        Part.SketchManager.InsertSketch(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Divide Surface into 3D Sketches Example (VBA)

This example:

- Shows how to divide the selected face into 3D sketches.
- Uses IModelDoc2::SketchConvertIsoCurves to extract ISO-parametric (UV) curves from a face and creates a set of evenly spaced sections in the U and V directions.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Select a face.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Generates 3D sketches.  
' 2. Evenly divides the 3D sketches on the selected  
'    face in the UV directions.  
' 3. Examine the graphics area, FeatureManager design  
'    tree, and the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Const nNum_U As Long = 3  
    Const nNum_V As Long = 4  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Debug.Print "  U percent ratios:"  
    For i = 0 To nNum_U  
        ' Start from 0%, finish at 100%  
        Debug.Print "    " & i * 100# / nNum_U & " %"  
        swModel.SketchConvertIsoCurves i * 100# / nNum_U, False, True, True  
    Next i  
  
    Debug.Print "  V percent ratios:"  
  
    For i = 0 To nNum_V  
        ' Start from 0%, finish at 100%  
        Debug.Print "    " & i * 100# / nNum_V & " %"  
        swModel.SketchConvertIsoCurves i * 100# / nNum_V, True, True, True  
    Next i  
  
End Sub
```

Obtain Version History Example (C++ COM)

This example shows how to obtain a version history of a file. IModelDoc2::IVersionHistory returns an array, so this code must be used in an in-process DLL.

```
DocumentVersionHistory(ISldWorks* m_pSldWorks)
{
    BSTR* versionHistStrings = NULL;
    LPMODELDOC2 pModelDoc = NULL;
    HRESULT hres= S_OK;

    hres = m_pSldWorks->get_IActiveDoc2(&pModelDoc);
    if (hres != S_OK || pModelDoc == NULL)
        return;

    int versHistCount = 0;
    hres = pModelDoc->IGetVersionHistoryCount(&versHistCount);
    if (hres != S_OK || versHistCount == 0)
    {
        pModelDoc->Release();
        return;
    }
    versionHistStrings = new BSTR[versHistCount];
    hres = pModelDoc->IVersionHistory(versionHistStrings);
    if (hres != S_OK || versionHistStrings == NULL)
    {
        delete [] versionHistStrings;
        pModelDoc->Release();
        return;
    }

    // For each version found
    for (int ii = 0; ii < versHistCount; ii++)
    {
        CString tempstr2(versionHistStrings[ii]);
        AfxMessageBox (tempstr2);

        SysFreeString( versionHistStrings[ii]);
    }

    // default destructor
    pModelDoc->Release();
    delete [] versionHistStrings;
}
```

Does Part Contain a Weldment Feature Example (VBA)

This example shows how to query a part document to find out if it contains a weldment feature.

```
'-----  
'  
' Preconditions: A model document containing a part is open.  
'  
' Postconditions: None  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swPartDoc As SldWorks.PartDoc  
Dim retVal As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swPartDoc = swApp.ActiveDoc  
retVal = swPartDoc.IsWeldment  
Debug.Print "Does this part contain a weldment feature?" & retVal  
  
End Sub
```

Drop Drawing Views from View Palette Example (VBA)

This example shows how to drop drawing views from the View Palette.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\blade shaft.sldasm.
' 2. Click File > Make Drawing from Assembly > OK.
' 3. Click a blank area of the FeatureManager design tree.
'
' Postconditions:
' 1. Drops the Current, Isometric, and Front views onto
'    the drawing sheet.
' 2. Examine the drawing and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Dim swModel As SldWorks.ModelDoc2
    Dim swDrawing As SldWorks.DrawingDoc
    Dim swFeature As SldWorks.Feature
    Dim swSubFeature As SldWorks.Feature
    Dim swSheet As SldWorks.Sheet
    Dim swView As SldWorks.View
    Dim bFound As Boolean
    Dim vViewNames As Variant
    Dim vViewName As Variant
    Dim strViewName As String
    Dim lNumViews As Long
    Dim dSheetScale As Double
    Dim vSheetProperties As Variant
    Dim nPaperSize As swDwgPaperSizes_e
    Dim dWidth As Double
    Dim dHeight As Double

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDrawing = swModel

    ' Get current sheet
    Set swSheet = swDrawing.GetCurrentSheet
    vSheetProperties = swSheet.GetProperties
    dSheetScale = vSheetProperties(2) / vSheetProperties(3)
    nPaperSize = swSheet.GetSize(dWidth, dHeight)

    Debug.Print "Sheet scale = " & dSheetScale
    Debug.Print "Sheet width = " & dWidth
    Debug.Print "Sheet height = " & dHeight

    lNumViews = 0
    vViewNames = swDrawing.GetDrawingPaletteViewNames

    If (Not (IsEmpty(vViewNames))) Then
        lNumViews = (UBound(vViewNames) - LBound(vViewNames) + 1)
        Debug.Print "Number of views on palette = " & lNumViews
        For Each vViewName In vViewNames
            strViewName = vViewName
            Debug.Print strViewName
            If (strViewName = "**Current") Then
                Set swView = swDrawing.DropDrawingViewFromPalette2(strViewName, dWidth / 5#, dHeight / 5#, 0#)
                Debug.Print " Dropped => " & swView.Name
            End If

            If (strViewName = "**Isometric") Then
                Set swView = swDrawing.DropDrawingViewFromPalette2(strViewName, dWidth / 3#, dHeight / 2#, 0#)
                Debug.Print " Dropped => " & swView.Name
            End If

            If (strViewName = "**Front") Then
                Set swView = swDrawing.DropDrawingViewFromPalette2(strViewName, dWidth / 4#, dHeight / 3#, 0#)
                Debug.Print " Dropped => " & swView.Name
            End If
        Next vViewName
    End If

End Sub

```

Drop First Drawing View from View Palette Example

This example shows how to drop the first drawing view on the View Palette onto the drawing.

```
'-----
'
' Preconditions: Part or assembly document
'             are open. An empty drawing
'             is open and active. Part or
'             assembly document is
'             selected in View Palette.
'
' Postconditions: The first view in the View
'               Palette is placed on
'               drawing sheet.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swDraw As DrawingDoc
Dim swNewView As View
Dim vPaletteNames As Variant

Sub main()

Set swApp = Application.SldWorks
Set swDraw = swApp.ActiveDoc

' Get the names of the drawing views on the View Palette
vPaletteNames = swDraw.GetDrawingPaletteViewNames()
Debug.Print "Names of the drawing views on the View Palette"
Debug.Print "before placing any on the drawing"
Debug.Print "===== "

' Print the names of the current views on the View Palette to the Immediate window
PrintPaletteNames vPaletteNames

' Get the names of the drawing before and after dropping it on the drawing
Debug.Print " "
Debug.Print "Drawing view to place: " & vPaletteNames(0)
Debug.Print " "
' Drop the drawing view
Set swNewView = swDraw.DropDrawingViewFromPalette2(vPaletteNames(0), 0.1, 0.1, 0.1)
Debug.Print "New drawing view name: " & swNewView.Name

' Print the names of the remaining views on the View Palette to the Immediate window
vPaletteNames = swDraw.GetDrawingPaletteViewNames()
Debug.Print " "
Debug.Print "Names of the drawing views remaining on the View Palette"
Debug.Print "after placing one on the drawing"
Debug.Print "===== "
PrintPaletteNames vPaletteNames

End Sub

' Print the names of the current views on the View Palette to the Immediate window
Public Sub PrintPaletteNames(vPaletteNames As Variant)
Dim index As Long
For index = 0 To UBound(vPaletteNames)
Debug.Print "View Palette view: " & vPaletteNames(index)
Next
End Sub
```

Duplicate, Delete, and Create Motion Study Example (C#)

This example shows how to create a duplicate motion study, delete an existing motion study, and create a new motion study.

```
//-----
// Preconditions:
// 1. Open an assembly document that has a motion study named
//    MotionStudy 1 and a collapsed exploded view.
// 2. Verify that a ray-trace rendering engine is active.
// 3. Add a reference to the SOLIDWORKS Motion Study primary interop
//    assembly (right-click the name of the project in the Project Explorer >
//    click Add Reference > browse to install_dir\api\redist ?
//    click SolidWorks.Interop.swmotionstudy.dll).
// 4. Verify that c:\test exists.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Observe the graphics area.
// 2. Makes a copy of Motion Study 1 called Motion Study 2.
// 3. Deletes and creates a new motion study for Motion Study 1.
// 4. Runs the animation (i.e., rotates, explodes,
//    stops rotating, and collapses the assembly).
// 5. Saves the motion study as c:\test\Anim1.avi.
// 6. Examine the Immediate window.
// 7. Play c:\test\Anim1.avi.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        private void PlayAnimation(MotionStudy SwMotionStudy)
        {
            Debug.Print("");
            int cPlayMode = 0;
            Debug.Print("Current play mode: ");
            cPlayMode = SwMotionStudy.PlayMode;

            //Get current play mode
            switch (cPlayMode)
            {
                case 1:
                    Debug.Print("    Normal");
                    break;
                case 2:
                    Debug.Print("    Loop");
                    break;
                case 3:
                    Debug.Print("    Reciprocate");
                    break;
            }

            //Set play mode to Loop
            int nPlayMode = 0;
            Debug.Print("New play mode: ");
            SwMotionStudy.PlayMode = (int)swAnimationPlayMode_e.swAnimationPlayModeLoop;
            nPlayMode = SwMotionStudy.PlayMode;

            //Get new play mode
            switch (nPlayMode)
            {
                case 1:
                    Debug.Print("    Normal");
                    break;
                case 2:
                    Debug.Print("    Loop");
                    break;
                case 3:
                    Debug.Print("    Reciprocate");
                    break;
            }

            //Set timebar to 0 on timeline
            SwMotionStudy.SetTime(0);

            //Play the animation
            SwMotionStudy.Play();
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MotionStudyManager swMotionMgr = default(MotionStudyManager);
            MotionStudy swMotionStudy1 = default(MotionStudy);
            AVIParameter swSaveAVIData = null;
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get the MotionManager
            swMotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();
        }
    }
}
```

```

if ((swMotionMgr == null))
{
    return;
}

//Get the motion study named Motion Study 1
swMotionStudy1 = (MotionStudy)swMotionMgr.GetMotionStudy("Motion Study 1");
if ((swMotionStudy1 == null))
{
    MessageBox.Show("Motion Study 1 is not available.");
    return;
}

//Create a copy of the motion study
swMotionStudy1.Duplicate();

//Get the supported motion study types
int MSTypes = 0;
boolstatus = swMotionStudy1.GetSupportedStudyTypes(out MSTypes);
Debug.Print("");
Debug.Print("Supported study types: ");
Debug.Print("    Assembly: " + ((MSTypes & (int)swMotionStudyType_e.swMotionStudyTypeAssembly) > 0));
Debug.Print("    PhysicalSimulation: " + ((MSTypes & (int)swMotionStudyType_e.swMotionStudyTypePhysicalSimulation) > 0));
Debug.Print("    CosmosMotion: " + ((MSTypes & (int)swMotionStudyType_e.swMotionStudyTypeCosmosMotion) > 0));
Debug.Print("    LegacyCosmosMotion: " + ((MSTypes & (int)swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion) > 0));

//Get the current motion study type
int CurStudyType = 0;
CurStudyType = swMotionStudy1.StudyType;
Debug.Print("");
Debug.Print("Current study type: ");
switch (CurStudyType)
{
    case (int)swMotionStudyType_e.swMotionStudyTypeAssembly:
        Debug.Print("    Assembly");
        break;
    case (int)swMotionStudyType_e.swMotionStudyTypePhysicalSimulation:
        Debug.Print("    PhysicalSimulation");
        break;
    case (int)swMotionStudyType_e.swMotionStudyTypeCosmosMotion:
        Debug.Print("    CosmosMotion");
        break;
    case (int)swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion:
        Debug.Print("    LegacyCosmosMotion");
        break;
}

//Is the motion study active? If not, activate it
if (!swMotionStudy1.IsActive)
    swMotionStudy1.Activate();

//Create an animation of the rotating model
//Delete any existing animation sequences
//Set the animation duration to 10 seconds
boolstatus = swMotionStudy1.CreateByRotateModel(true, (int)swAnimatorAxisOfRotation_e.swRotationAboutYAxis, 1, (int)swAnimatorDirectionOfRotation_e.swRotationClockwise, 10, 0);

//Play the animation
PlayAnimation(swMotionStudy1);

//Stop playing the animation
swMotionStudy1.Stop();

//Add an explode to the animation
//Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByExplode(false, 5, 0);

//Add a collapse to the animation
//Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByCollapse(false, 10, 5);

//Set duration of animation to 15 seconds
swMotionStudy1.SetDuration(15);

//Play the animation
PlayAnimation(swMotionStudy1);

//Calculate
swMotionStudy1.Calculate();
Debug.Print("");
Debug.Print("Study duration: " + swMotionStudy1.GetDuration());

//Play animation
PlayAnimation(swMotionStudy1);

//Set and save AVI parameters
swSaveAVIData = (AVIParameter)swMotionMgr.CreateAVIParameter();
swSaveAVIData.FramePerSecond = 7.5;
swSaveAVIData.SaveEntireAnimation = true;
swSaveAVIData.OutputType = (int)swAnimationOutputType_e.swAnimationOutput_AVI;
swSaveAVIData.RendererType = (int)swRendererType_e.swRendererType_Solidworks_Screen;
swSaveAVIData.MotionBlur = true;
swSaveAVIData.BlurLength = 10;
swSaveAVIData.BlurOffset = -100;
swMotionStudy1.Stop();

//Save animation as .avi file
swMotionStudy1.SaveToAVI("C:\\test\\Anim1.avi", swSaveAVIData);
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Duplicate, Delete, and Create Motion Study Example (VBA)

This example shows how to create a duplicate motion study, delete an existing motion study, and create a new motion study.

```
'-----
' Preconditions:
' 1. Open an assembly document that has a motion study named
'    MotionStudy 1 and a collapsed exploded view.
' 2. Verify that a ray-trace rendering engine is active.
' 3. Add a reference to the SOLIDWORKS MotionStudy type library.
' 4. Verify that c:\test exists.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Observe the graphics area.
' 2. Makes a copy of Motion Study 1 called Motion Study 2.
' 3. Deletes and creates a new motion study for Motion Study 1.
' 4. Runs the animation (i.e., rotates, explodes,
'    stops rotating, and collapses the assembly).
' 5. Saves the motion study as c:\test\Anim1.avi.
' 6. Examine the Immediate window.
' 7. Play c:\test\Anim1.avi.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMotionMgr As SwMotionStudy.MotionStudyManager
Dim swMotionStudy1 As SwMotionStudy.MotionStudy
Dim swSaveAVIData As SwMotionStudy.AVIParameter
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    'Get the motion study named Motion Study 1
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")
    If (swMotionStudy1 Is Nothing) Then
        MsgBox "Motion Study 1 is not available."
        End
    End If

    'Create a copy of the motion study
    swMotionStudy1.Duplicate

    'Get the supported motion study types
    Dim MStypes As Long
    boolstatus = swMotionStudy1.GetSupportedStudyTypes(MStypes)
    Debug.Print ""
    Debug.Print "Supported study types: "
    Debug.Print "    Assembly: " & ((MStypes And swMotionStudyType_e.swMotionStudyTypeAssembly) > 0)
    Debug.Print "    PhysicalSimulation: " & ((MStypes And swMotionStudyType_e.swMotionStudyTypePhysicalSimulation) > 0)
    Debug.Print "    CosmosMotion: " & ((MStypes And swMotionStudyType_e.swMotionStudyTypeCosmosMotion) > 0)
    Debug.Print "    LegacyCosmosMotion: " & ((MStypes And swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion) > 0)

    'Get the current motion study type
    Dim CurStudyType As Long
    CurStudyType = swMotionStudy1.StudyType
    Debug.Print ""
    Debug.Print "Current study type: "
    Select Case CurStudyType
        Case swMotionStudyType_e.swMotionStudyTypeAssembly
            Debug.Print "    Assembly"
        Case swMotionStudyType_e.swMotionStudyTypePhysicalSimulation
            Debug.Print "    PhysicalSimulation"
        Case swMotionStudyType_e.swMotionStudyTypeCosmosMotion
            Debug.Print "    CosmosMotion"
        Case swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion
            Debug.Print "    LegacyCosmosMotion"
    End Select

    'Is the motion study active? If not, activate it
    If Not swMotionStudy1.IsActive Then swMotionStudy1.Activate

    'Create an animation of the rotating model
    'Delete any existing animation sequences
    'Set the animation duration to 10 seconds
    boolstatus = swMotionStudy1.CreateByRotateModel(True, swAnimatorAxisOfRotation_e.swRotationAboutYAxis, 1,
    swAnimatorDirectionOfRotation_e.swRotationClockwise, 10, 0)

    'Play the animation
    PlayAnimation swMotionStudy1

    'Stop playing the animation
    swMotionStudy1.Stop
```

```

'Add an explode to the animation
'Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByExplode(False, 5, 0)

'Add a collapse to the animation
'Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByCollapse(False, 10, 5)

'Set duration of animation to 15 seconds
swMotionStudy1.SetDuration (15)

'Play the animation
PlayAnimation swMotionStudy1

'Calculate
swMotionStudy1.Calculate

Debug.Print ""
Debug.Print "Study duration: " & swMotionStudy1.GetDuration

'Play animation
PlayAnimation swMotionStudy1

'Set and save AVI parameters
Set swSaveAVIData = swMotionMgr.CreateAVIParameter()
swSaveAVIData.FramePerSecond = 7.5
swSaveAVIData.SaveEntireAnimation = True
swSaveAVIData.OutputType = swAnimationOutputType_e.swAnimationOutput_AVI
swSaveAVIData.RendererType = swRendererType_e.swRendererType_Solidworks_Screen
swSaveAVIData.MotionBlur = True
swSaveAVIData.BlurLength = 10
swSaveAVIData.BlurOffset = -100

swMotionStudy1.Stop

'Save animation as .avi file
swMotionStudy1.SaveToAVI "C:\test\Anim1.avi", swSaveAVIData

End Sub

Private Sub PlayAnimation(SwMotionStudy As SwMotionStudy.MotionStudy)
Debug.Print ""
Dim cPlayMode As Long
Debug.Print "Current play mode: "
cPlayMode = SwMotionStudy.PlayMode

'Get current play mode
Select Case cPlayMode
    Case 1
        Debug.Print "    Normal"
    Case 2
        Debug.Print "    Loop"
    Case 3
        Debug.Print "    Reciprocate"
End Select

'Set play mode to Loop
Dim nPlayMode As Long
Debug.Print "New play mode: "
SwMotionStudy.PlayMode = swAnimationPlayMode_e.swAnimationPlayModeLoop
nPlayMode = SwMotionStudy.PlayMode
'Get new play mode
Select Case nPlayMode
    Case 1
        Debug.Print "    Normal"
    Case 2
        Debug.Print "    Loop"
    Case 3
        Debug.Print "    Reciprocate"
End Select

'Set timebar to 0 on timeline
SwMotionStudy.SetTime 0

'Play the animation
SwMotionStudy.Play

End Sub

```

Duplicate, Delete, and Create Motion Study Example (VB.NET)

This example shows how to create a duplicate motion study, delete an existing motion study, and create a new motion study.

Preconditions:

1. Open an assembly document that has a motion study named **MotionStudy 1** and a collapsed exploded view.
2. Verify that a ray-trace rendering engine is active.
3. Add a reference to the SOLIDWORKS Motion Study primary interop assembly (right-click the name of the project in the Project Explorer > click **Add Reference** > browse to *install_dir\api\redist* > click **SolidWorks.Interop.swmotionstudy.dll**).
4. Verify that *c:\test* exists.
5. Open the Immediate window.

Postconditions:

1. Observe the graphics area.
2. Makes a copy of **Motion Study 1** called **Motion Study 2**.
3. Deletes and creates a new motion study for **Motion Study 1**.
4. Runs the animation (i.e., rotates, explodes, stops rotating, and collapses the assembly).
5. Saves the motion study as *c:\test\Anim1.avi*.
6. Examine the Immediate window.
7. Play *c:\test\Anim1.avi*.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Private Sub PlayAnimation(ByVal SwMotionStudy As MotionStudy)
        Debug.Print("")
        Dim cPlayMode As Integer
        Debug.Print("Current play mode: ")
        cPlayMode = SwMotionStudy.PlayMode

        'Get current play mode
        Select Case cPlayMode
            Case 1
                Debug.Print("    Normal")
            Case 2
                Debug.Print("    Loop")
            Case 3
                Debug.Print("    Reciprocate")
        End Select

        'Set play mode to Loop
        Dim nPlayMode As Integer
        Debug.Print("New play mode: ")
        SwMotionStudy.PlayMode = swAnimationPlayMode_e.swAnimationPlayModeLoop
        nPlayMode = SwMotionStudy.PlayMode

        'Get new play mode
        Select Case nPlayMode
            Case 1
                Debug.Print("    Normal")
            Case 2
                Debug.Print("    Loop")
            Case 3
                Debug.Print("    Reciprocate")
        End Select

        'Set timebar to 0 on timeline
        SwMotionStudy.SetTime(0)

        'Play the animation
        SwMotionStudy.Play()
    End Sub

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMotionMgr As MotionStudyManager
        Dim swMotionStudy1 As MotionStudy
        Dim swSaveAVIData As AVIPParameter = Nothing
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Get the MotionManager
        swMotionMgr = swModelDocExt.GetMotionStudyManager()
        If (swMotionMgr Is Nothing) Then
            Return
        End If

        'Get the motion study named Motion Study 1
        swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")
        If (swMotionStudy1 Is Nothing) Then
            MsgBox("Motion Study 1 is not available.")
            Return
        End If

        'Create a copy of the motion study
        swMotionStudy1.Duplicate()

        'Get the supported motion study types
        Dim MSTypes As Integer
        boolstatus = swMotionStudy1.GetSupportedStudyTypes(MSTypes)
        Debug.Print("")
        Debug.Print("Supported study types: ")
        Debug.Print("    Assembly: " & ((MSTypes And swMotionStudyType_e.swMotionStudyTypeAssembly) > 0))
        Debug.Print("    PhysicalSimulation: " & ((MSTypes And swMotionStudyType_e.swMotionStudyTypePhysicalSimulation) > 0))
    End Sub
```

```

Debug.Print("    CosmosMotion: " & ((MSTypes And swMotionStudyType_e.swMotionStudyTypeCosmosMotion) > 0))
Debug.Print("    LegacyCosmosMotion: " & ((MSTypes And swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion) > 0))

'Get the current motion study type
Dim CurStudyType As Integer
CurStudyType = swMotionStudy1.StudyType
Debug.Print("")
Debug.Print("Current study type: ")
Select Case CurStudyType
    Case swMotionStudyType_e.swMotionStudyTypeAssembly
        Debug.Print("    Assembly")
    Case swMotionStudyType_e.swMotionStudyTypePhysicalSimulation
        Debug.Print("    PhysicalSimulation")
    Case swMotionStudyType_e.swMotionStudyTypeCosmosMotion
        Debug.Print("    CosmosMotion")
    Case swMotionStudyType_e.swMotionStudyTypeLegacyCosmosMotion
        Debug.Print("    LegacyCosmosMotion")
End Select

'Is the motion study active? If not, activate it
If Not swMotionStudy1.IsActive Then swMotionStudy1.Activate()

'Create an animation of the rotating model
'Delete any existing animation sequences
'Set the animation duration to 10 seconds
boolstatus = swMotionStudy1.CreateByRotateModel(True, swAnimatorAxisOfRotation_e.swRotationAboutYAxis, 1, swAnimatorDirectionOfRotation_e.swRotati
onClockwise, 10, 0)

'Play the animation
PlayAnimation(swMotionStudy1)

'Stop playing the animation
swMotionStudy1.Stop()

'Add an explode to the animation
'Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByExplode(False, 5, 0)

'Add a collapse to the animation
'Set the animation duration to 5 seconds
boolstatus = swMotionStudy1.CreateByCollapse(False, 10, 5)

'Set duration of animation to 15 seconds
swMotionStudy1.SetDuration(15)

'Play the animation
PlayAnimation(swMotionStudy1)

'Calculate
swMotionStudy1.Calculate()
Debug.Print("")
Debug.Print("Study duration: " & swMotionStudy1.GetDuration)

'Play animation
PlayAnimation(swMotionStudy1)

'Set and save AVI parameters
swSaveAVIData = swMotionMgr.CreateAVIParameter()
swSaveAVIData.FramePerSecond = 7.5
swSaveAVIData.SaveEntireAnimation = True
swSaveAVIData.OutputType = swAnimationOutputType_e.swAnimationOutput_AVI
swSaveAVIData.RendererType = swRendererType_e.swRendererType_Solidworks_Screen
swSaveAVIData.MotionBlur = True
swSaveAVIData.BlurLength = 10
swSaveAVIData.BlurOffset = -100
swMotionStudy1.Stop()

'Save animation as .avi file
swMotionStudy1.SaveToAVI("C:\test\Anim1.avi", swSaveAVIData)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Dynamically Rotate Model View Example (C++ COM)

This example shows how to dynamically rotate a model view. It also shows you how to speed up the process by using the StartDynamics and StopDynamics calls.

```
RotateTheView(ISldWorks* m_pSldWorks)
{
    LPMODELDOC2 p_ModelDoc = NULL;
    LPMODELVIEW p_ModelView = NULL;
    HRESULT res = S_OK;
    // Retrieve the model document pointer
    res = m_pSldWorks->get_IActiveDoc2( &p_ModelDoc );
    if( res != S_OK || p_ModelDoc == NULL )
        return;
    // Get the active view pointer
    res = p_ModelDoc->get_IActiveView(&p_ModelView);
    if( res != S_OK || p_ModelView == NULL )
    {
        p_ModelDoc->Release();
        return;
    }
    int i;
    // Prepare view for rotation
    p_ModelView->StartDynamics();
    for (i = 1; i < 100; i++)
    {
        p_ModelDoc->ViewRotateplusy();
    }
    // End dynamic rotation mode
    p_ModelView->StopDynamics();
    // Repaint the screen
    p_ModelDoc->GraphicsRedraw2();
    //Clean up
    p_ModelView->Release();
    p_ModelDoc->Release();
}
```

Dynamically Mirror Sketch Entities Example (VBA)

This example shows how to enable dynamic sketch mirroring.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldprt.  
' 2. Select the top planar face on the part and open a sketch.  
' 3. Select an edge on the planar face selected in step 2.  
'  
' Postconditions:  
' 1. Changes sketch mode to dynamic sketch mirror mode.  
' 2. Creates a line and mirrors that line about the edge selected  
'    in Preconditions step 3.  
' 3. Examine the graphics area.  
'  
' NOTE: Because this part is used elsewhere, do  
' not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSkMgr As SldWorks.SketchManager  
Dim swSketch As SldWorks.Sketch  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swSelMgr As SldWorks.SelectionMgr  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSkMgr = swModel.SketchManager  
Set swSelMgr = swModel.SelectionManager  
  
swSkMgr.SetDynamicMirror (True)  
  
Set swSketch = swModel.GetActiveSketch2  
Set swSketchSegment = swModel.CreateLine2(0, 0, 0, 1, 1, 1)  
  
End Sub
```

Edit Balloon Example (C#)

This example shows how to edit a balloon in a drawing document.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Click Insert > Annotations > Balloon.
// 3. Click a model edge in either drawing view and add the balloon.
// 4. Close the Balloon PropertyManager page.
// 5. Select the balloon in the drawing.
//
// Postconditions: The properties of the selected balloon are modified.
//
// NOTE: Because this drawing document is used elsewhere, do not save any
// changes when closing it.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace EditBalloonProperties_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;
        Note swNote;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Get the selected balloon
            swNote = (Note)swSelMgr.GetSelectedObject6(1, -1);

            // Edit the selected balloon
            swNote = (Note)swModelDocExt.EditBalloonProperties2((int)swBalloonStyle_e.swBS_SplitCirc,
(int)swBalloonFit_e.swBF_5Chars, (int)swBalloonTextContent_e.swBalloonTextCustom, "Upper",
(int)swBalloonTextContent_e.swBalloonTextCustom, "Lower", 0, true, 1, "X",
0.0355);

            Debug.Print("Balloon name: " + swNote.GetName());
        }

        public SldWorks swApp;
    }
}
```

Edit Balloon Example (VBA)

This example shows how to edit a balloon in a drawing document.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Click Insert > Annotations > Balloon.  
' 3. Click a model edge in either drawing view and add the balloon.  
' 4. Close the Balloon PropertyManager page.  
' 5. Select the balloon in the drawing.  
' -----  
' Postconditions: The properties of the selected balloon are modified.  
' -----  
' NOTE: Because this drawing document is used elsewhere, do not save any  
' changes when closing it.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swNote As SldWorks.Note  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Get the selected balloon  
    Set swNote = swSelMgr.GetSelectedObject6(1, -1)  
  
    ' Edit the selected balloon  
    Set swNote = swModelDocExt.EditBalloonProperties2(swBS_SplitCirc, swBF_5Chars, swBalloonTextCustom, "Upper",  
    swBalloonTextCustom, "Lower", 0, True, 1, "X", 0.0355)  
  
    Debug.Print "Balloon name: " & swNote.GetName  
End Sub
```

Edit Balloon Example (VB.NET)

This example shows how to edit a balloon in a drawing document.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Click Insert > Annotations > Balloon.
' 3. Click a model edge in either drawing view and add the balloon.
' 4. Close the Balloon PropertyManager page.
' 5. Select the balloon in the drawing.
'-----
' Postconditions: The properties of the selected balloon are modified.
'
' NOTE: Because this drawing document is used elsewhere, do not save any
' changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim swNote As Note

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Get the selected balloon
        swNote = swSelMgr.GetSelectedObject6(1, -1)

        ' Edit the selected balloon
        swNote = swModelDocExt.EditBalloonProperties2(swBalloonStyle_e.swBS_SplitCirc, swBalloonFit_e.swBF_5Chars,
        swBalloonTextContent_e.swBalloonTextCustom, "Upper", swBalloonTextContent_e.swBalloonTextCustom, "Lower", 0, True, 1, "X", 0.0355)

        Debug.Print("Balloon name: " & swNote.GetName)

    End Sub

    Public swApp As SldWorks

End Class

```

Edit Mate Example (C#)

This example shows how to edit an assembly mate.

```
//-----
// Preconditions:
// 1. Open an assembly with a mate.
// 2. Select a mate.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Edits the selected assembly mate.
// 2. Examine the Immediate window.
//
// Note: Modifying a mate can cause mate errors.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public bool SelectMateEntity(SldWorks swApp, ModelDoc2 swModel, MateEntity2 swMateEnt, int nMark)
        {
            bool functionReturnValue = false;

            Entity swEnt = default(Entity);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            bool bRet = false;

            switch (swMateEnt.ReferenceType)
            {
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Point:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Line:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Circle:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Plane:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Cylinder:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Sphere:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Cone:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_SweptSurface:

                    swSelMgr = (SelectionMgr)swModel.SelectionManager;
                    swSelData = (SelectData)swSelMgr.CreateSelectData();
                    swEnt = (Entity)swMateEnt.Reference;

                    swSelData.Mark = nMark;

                    bRet = swEnt.Select4(true, swSelData);

                    functionReturnValue = bRet;

                    return functionReturnValue;

                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Set:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_MultipleSurface:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_GenSurface:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Ellipse:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_GeneralCurve:
                case (int)swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_UNKNOWN:

                    break;

                default:

                    break;
            }

            functionReturnValue = false;
            return functionReturnValue;
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
        }
    }
}
```

```

SelectionMgr swSelMgr = default(SelectionMgr);
Feature swFeat = default(Feature);
Mate2 swMate = default(Mate2);
DisplayDimension swDispDim = default(DisplayDimension);
Dimension swDim = default(Dimension);
string sVarType = null;
double nVarFactor = 0;
double nMateDist = 0;
int nNumMateEnt = 0;
MateEntity2 swMateEnt = null; ;
double[] vMateEntPar = null;
Component2 swComp = default(Component2);
int nNewMateAlign = 0;
int nRetVal = 0;
int i = 0;
bool bRet = false;
double[] vDimValueArr = null;

swModel = (ModelDoc2)swApp.ActiveDoc;
swAssy = (AssemblyDoc)swModel;
swSelMgr = (SelectionMgr)swModel.SelectionManager;
swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
swMate = (Mate2)swFeat.GetSpecificFeature2();
swDispDim = swMate.get_DisplayDimension2(0);

Debug.Print("File = " + swModel.GetPathName());
Debug.Print(" " + swFeat.Name);
Debug.Print("      Type      = " + swMate.Type);
Debug.Print("      Alignment  = " + swMate.Alignment);
Debug.Print("      CanBeFlipped = " + swMate.CanBeFlipped);

switch (swMate.Type)
{
    case (int)swMateType_e.swMateANGLE:

        sVarType = " deg";
        // radian = 180°/p = 57.295779513° or approximately 57.3°
        nVarFactor = 57.3;

        break;
    case (int)swMateType_e.swMateDISTANCE:

        sVarType = " mm";
        nVarFactor = 1000.0;

        break;
    case (int)swMateType_e.swMateGEAR:

        sVarType = " ratio";
        nVarFactor = 1.0;

        break;
}

if ((int)swMateType_e.swMateANGLE == swMate.Type | (int)swMateType_e.swMateDISTANCE == swMate.Type)
{
    Debug.Print("      MaxVar      = " + swMate.MaximumVariation * nVarFactor + sVarType);
    Debug.Print("      MinVar      = " + swMate.MinimumVariation * nVarFactor + sVarType);
}

if ((swDispDim != null))
{
    swDim = (Dimension)swDispDim.GetDimension();
    vDimValueArr = (double[])swDim.GetSystemValue3((int)swInConfigurationOpts_e.swThisConfiguration, null);
    Debug.Print("      Dim Value    = " + vDimValueArr[0] * nVarFactor + sVarType);
}

nNumMateEnt = swMate.GetMateEntityCount();

for (i = 0; i < nNumMateEnt; i++)
{
    swMateEnt = swMate.MateEntity(i);
    swComp = swMateEnt.ReferenceComponent;

    vMateEntPar = (double[])swMateEnt.EntityParams;

    Debug.Print("      RefType(" + i + ") = " + swMateEnt.ReferenceType);
    Debug.Print("      Component          = " + swComp.Name2 + " (" + swComp.ReferencedConfiguration + ") --> " +
swComp.GetPathName());
    Debug.Print("      Point              = (" + vMateEntPar[0] * 1000.0 + ", " + vMateEntPar[1] * 1000.0 + ", " +
vMateEntPar[2] * 1000.0 + ") mm");
    Debug.Print("      Vector             = (" + vMateEntPar[3] + ", " + vMateEntPar[4] + ", " + vMateEntPar[5] +
");");
    Debug.Print("      Radius 1           = " + vMateEntPar[6] * 1000.0 + " mm");
    Debug.Print("      Radius 2           = " + vMateEntPar[7] * 1000.0 + " mm");
}

switch (swMate.Type)
{

```

```

        case (int)swMateType_e.swMateGEAR:
            // Cannot change alignment on these mate types
            return;
    }

    if ((int)swMateAlign_e.swMateAlignALIGNED == swMate.Alignment)
    {
        nNewMateAlign = (int)swMateAlign_e.swMateAlignANTI_ALIGNED;
    }
    else
    {
        if ((int)swMateAlign_e.swMateAlignANTI_ALIGNED == swMate.Alignment)
        {
            nNewMateAlign = (int)swMateAlign_e.swMateAlignALIGNED;
        }
        else
        {
            return;
        }
    }
}

swModel.ClearSelection2(true);

for (i = 0; i <= nNumMateEnt - 1; i++)
{
    // IAssemblyDoc::EditMate3 requires that mate entities
    // be selected with mark of 1 except for:
    //     swMateCAMFOLLOWER
    //         cam face          --> 1
    //         cam follower face --> 8
    //     swMateSYMMETRIC
    //         symmetry faces    --> 1
    //         symmetry plane    --> 4

    bRet = SelectMateEntity(swApp, swModel, swMate.MateEntity(i), 1);
}

// IAssemblyDoc::EditMate3 requires mate feature to be selected last
// Mark is ignored
bRet = swFeat.Select2(true, 0);

swAssy.EditMate3(swMate.Type, nNewMateAlign, true, nMateDist, swMate.MaximumVariation, swMate.MinimumVariation, 0.0,
0.0, nMateDist, swMate.MaximumVariation,
swMate.MinimumVariation, false, true, 0, out nRetVal);

bRet = swModel.EditRebuild3();
}

public SldWorks swApp;
}
}

```

Edit Mate Example (VBA)

This example shows how to edit an assembly mate.

```

'-----
' Preconditions:
' 1. Open an assembly with a mate.
' 2. Select a mate.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Edits the selected assembly mate.
' 2. Examine the Immediate window.
'
' NOTE: Modifying a mate can cause mate errors.
'-----

```

Option Explicit

```
Function SelectMateEntity(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swMateEnt As SldWorks.MateEntity2, nMark As Long) As Boolean
```

```

    Dim swEnt As SldWorks.Entity
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim bRet As Boolean

    Select Case swMateEnt.ReferenceType
        Case swMateEntity2ReferenceType_Point, _
            swMateEntity2ReferenceType_Line, _
            swMateEntity2ReferenceType_Circle, _
            swMateEntity2ReferenceType_Plane, _
            swMateEntity2ReferenceType_Cylinder, _
            swMateEntity2ReferenceType_Sphere, _
            swMateEntity2ReferenceType_Cone, _
            swMateEntity2ReferenceType_SweptSurface

            Set swSelMgr = swModel.SelectionManager
            Set swSelData = swSelMgr.CreateSelectData
            Set swEnt = swMateEnt.Reference

            swSelData.Mark = nMark

            bRet = swEnt.Select4(True, swSelData)

            SelectMateEntity = bRet

            Exit Function

        Case swMateEntity2ReferenceType_Set, _
            swMateEntity2ReferenceType_MultipleSurface, _
            swMateEntity2ReferenceType_GenSurface, _
            swMateEntity2ReferenceType_Ellipse, _
            swMateEntity2ReferenceType_GeneralCurve, _
            swMateEntity2ReferenceType_UNKNOWN

        Case Else
    End Select

    SelectMateEntity = False

```

```
End Function
```

```
Sub main()
```

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swMate As SldWorks.Mate2
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swDim As SldWorks.Dimension
    Dim sVarType As String
    Dim nVarFactor As Double
    Dim nMateDist As Double
    Dim nNumMateEnt As Long
    Dim swMateEnt() As SldWorks.MateEntity2
    Dim vMateEntPar As Variant
    Dim swComp As SldWorks.Component2
    Dim nNewMateAlign As Long
    Dim nRetVal As Long
    Dim i As Long
    Dim bRet As Boolean
    Dim vDimValueArr As Variant

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swMate = swFeat.GetSpecificFeature2
    Set swDispDim = swMate.DisplayDimension2(0)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name

```

```

Debug.Print "    Type           = " & swMate.Type
Debug.Print "    Alignment        = " & swMate.Alignment
Debug.Print "    CanBeFlipped    = " & swMate.CanBeFlipped
Select Case swMate.Type
    Case swMateANGLE
        sVarType = " deg"
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        nVarFactor = 57.3
    Case swMateDISTANCE
        sVarType = " mm"
        nVarFactor = 1000#
    Case swMateGEAR
        sVarType = " ratio"
        nVarFactor = 1#
End Select
If swMateANGLE = swMate.Type Or swMateDISTANCE = swMate.Type Then
    Debug.Print "    MaxVar           = " & swMate.MaximumVariation * nVarFactor & sVarType
    Debug.Print "    MinVar           = " & swMate.MinimumVariation * nVarFactor & sVarType
End If
If Not swDispDim Is Nothing Then
    Set swDim = swDispDim.GetDimension
    vDimValueArr = swDim.GetSystemValue3(swThisConfiguration, Empty)
    Debug.Print "    Dim Value        = " & vDimValueArr(0) * nVarFactor & sVarType
End If
nNumMateEnt = swMate.GetMateEntityCount
ReDim swMateEnt(nNumMateEnt)
For i = 0 To nNumMateEnt - 1
    Set swMateEnt(i) = swMate.MateEntity(i)
    Set swComp = swMateEnt(i).ReferenceComponent
    vMateEntPar = swMateEnt(i).EntityParams
    Debug.Print "    RefType(" & i & ") = " & swMateEnt(i).ReferenceType
    Debug.Print "    Component         = " & swComp.Name2 & " (" & swComp.ReferencedConfiguration & ") --> " &
swComp.GetPathName
    Debug.Print "    Point             = (" & vMateEntPar(0) * 1000# & ", " & vMateEntPar(1) * 1000# & ", " &
vMateEntPar(2) * 1000# & ") mm"
    Debug.Print "    Vector            = (" & vMateEntPar(3) & ", " & vMateEntPar(4) & ", " & vMateEntPar(5) & ")"
    Debug.Print "    Radius 1         = " & vMateEntPar(6) * 1000# & " mm"
    Debug.Print "    Radius 2         = " & vMateEntPar(7) * 1000# & " mm"
Next i
Select Case swMate.Type
    Case swMateGEAR
        ' Cannot change alignment on these mate types
        Exit Sub
End Select
If swMateAlignALIGNED = swMate.Alignment Then
    nNewMateAlign = swMateAlignANTI_ALIGNED
Else
    If swMateAlignANTI_ALIGNED = swMate.Alignment Then
        nNewMateAlign = swMateAlignALIGNED
    Else
        Exit Sub
    End If
End If
swModel.ClearSelection2 True
For i = 0 To nNumMateEnt - 1
    ' IAssemblyDoc::EditMate3 requires that mate entities
    ' be selected with mark of 1 except for:
    '     swMateCAMFOLLOWER
    '         cam face          --> 1
    '         cam follower face --> 8
    '     swMateSYMMETRIC
    '         symmetry faces    --> 1
    '         symmetry plane    --> 4
    bRet = SelectMateEntity(swApp, swModel, swMateEnt(i), 1)
Next i
' IAssemblyDoc::EditMate3 requires mate feature to be selected last
' Mark is ignored
bRet = swFeat.Select2(True, 0)
swAssy.EditMate3 _
    swMate.Type, _
    nNewMateAlign, _
    True, _
    nMateDist, _
    swMate.MaximumVariation, _
    swMate.MinimumVariation, _

```

```
0#, _  
0#, _  
nMateDist, _  
swMate.MaximumVariation, _  
swMate.MinimumVariation, _  
False, _  
True, _  
0, _  
nRetVal  
bRet = swModel.EditRebuild3  
End Sub
```

Edit Mate Example (VB.NET)

This example shows how to edit an assembly mate.

```
'-----
' Preconditions:
' 1. Open an assembly with a mate.
' 2. Select a mate.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Edits the selected assembly mate.
' 2. Examine the Immediate window.
'
' Note: Modifying a mate can cause mate errors.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Function SelectMateEntity(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swMateEnt As MateEntity2, ByVal nMark As Integer) As Boolean
```

```
        Dim swEnt As Entity
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim bRet As Boolean
```

```
        Select Case swMateEnt.ReferenceType
```

```
            Case swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Point, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Line, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Circle, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Plane, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Cylinder, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Sphere, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Cone, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_SweptSurface
```

```
                swSelMgr = swModel.SelectionManager
                swSelData = swSelMgr.CreateSelectData
                swEnt = swMateEnt.Reference
```

```
                swSelData.Mark = nMark
```

```
                bRet = swEnt.Select4(True, swSelData)
```

```
                SelectMateEntity = bRet
```

```
            Exit Function
```

```
            Case swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Set, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_MultipleSurface, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_GenSurface, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_Ellipse, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_GeneralCurve, _
                swMateEntity2ReferenceType_e.swMateEntity2ReferenceType_UNKNOWN
```

```
            Case Else
```

```
        End Select
```

```
        SelectMateEntity = False
```

```
    End Function
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swMate As Mate2
        Dim swDispDim As DisplayDimension
        Dim swDim As Dimension
        Dim sVarType As String = Nothing
        Dim nVarFactor As Double
        Dim nMateDist As Double
        Dim nNumMateEnt As Integer
        Dim swMateEnt() As MateEntity2
        Dim vMateEntPar As Object
        Dim swComp As Component2
        Dim nNewMateAlign As Integer
        Dim nRetVal As Integer
        Dim i As Integer
```

```

Dim bRet As Boolean
Dim vDimValueArr As Object

swModel = swApp.ActiveDoc
swAssy = swModel
swSelMgr = swModel.SelectionManager
swFeat = swSelMgr.GetSelectedObject6(1, -1)
swMate = swFeat.GetSpecificFeature2
swDispDim = swMate.DisplayDimension2(0)

Debug.Print("File = " & swModel.GetPathName)
Debug.Print(" " & swFeat.Name)
Debug.Print("    Type = " & swMate.Type)
Debug.Print("    Alignment = " & swMate.Alignment)
Debug.Print("    CanBeFlipped = " & swMate.CanBeFlipped)

Select Case swMate.Type

    Case swMateType_e.swMateANGLE

        sVarType = " deg"
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        nVarFactor = 57.3

    Case swMateType_e.swMateDISTANCE

        sVarType = " mm"
        nVarFactor = 1000.0#

    Case swMateType_e.swMateGEAR

        sVarType = " ratio"
        nVarFactor = 1.0#

End Select

If swMateType_e.swMateANGLE = swMate.Type Or swMateType_e.swMateDISTANCE = swMate.Type Then

    Debug.Print("    MaxVar = " & swMate.MaximumVariation * nVarFactor & sVarType)
    Debug.Print("    MinVar = " & swMate.MinimumVariation * nVarFactor & sVarType)

End If

If Not swDispDim Is Nothing Then

    swDim = swDispDim.GetDimension
    vDimValueArr = swDim.GetSystemValue3(swInConfigurationOpts_e.swThisConfiguration, Nothing)
    Debug.Print("    Dim Value = " & vDimValueArr(0) * nVarFactor & sVarType)

End If

nNumMateEnt = swMate.GetMateEntityCount

ReDim swMateEnt(nNumMateEnt)

For i = 0 To nNumMateEnt - 1

    swMateEnt(i) = swMate.MateEntity(i)
    swComp = swMateEnt(i).ReferenceComponent

    vMateEntPar = swMateEnt(i).EntityParams

    Debug.Print("    RefType(" & i & ") = " & swMateEnt(i).ReferenceType)
    Debug.Print("    Component = " & swComp.Name2 & " (" & swComp.ReferencedConfiguration & ") --> " &
swComp.GetPathName)
    Debug.Print("    Point = (" & vMateEntPar(0) * 1000.0# & ", " & vMateEntPar(1) * 1000.0# & ", " &
vMateEntPar(2) * 1000.0# & ") mm")
    Debug.Print("    Vector = (" & vMateEntPar(3) & ", " & vMateEntPar(4) & ", " & vMateEntPar(5) & ")")
    Debug.Print("    Radius 1 = " & vMateEntPar(6) * 1000.0# & " mm")
    Debug.Print("    Radius 2 = " & vMateEntPar(7) * 1000.0# & " mm")

Next i

Select Case swMate.Type

    Case swMateType_e.swMateGEAR
        ' Cannot change alignment on these mate types
        Exit Sub

End Select

If swMateAlign_e.swMateAlignALIGNED = swMate.Alignment Then
    nNewMateAlign = swMateAlign_e.swMateAlignANTI_ALIGNED
Else
    If swMateAlign_e.swMateAlignANTI_ALIGNED = swMate.Alignment Then
        nNewMateAlign = swMateAlign_e.swMateAlignALIGNED
    Else
        Exit Sub
    End If
End If

End If

```

```

swModel.ClearSelection2(True)

For i = 0 To nNumMateEnt - 1
    ' IAssemblyDoc::EditMate3 requires that mate entities
    ' be selected with mark of 1 except for:
    '     swMateCAMFOLLOWER
    '         cam face          --> 1
    '         cam follower face --> 8
    '     swMateSYMMETRIC
    '         symmetry faces    --> 1
    '         symmetry plane    --> 4

    bRet = SelectMateEntity(swApp, swModel, swMateEnt(i), 1)

Next i

' IAssemblyDoc::EditMate3 requires mate feature to be selected last
' Mark is ignored
bRet = swFeat.Select2(True, 0)

swAssy.EditMate3( _
    swMate.Type, _
    nNewMateAlign, _
    True, _
    nMateDist, _
    swMate.MaximumVariation, _
    swMate.MinimumVariation, _
    0.0#, _
    0.0#, _
    nMateDist, _
    swMate.MaximumVariation, _
    swMate.MinimumVariation, _
    False, _
    True, _
    0, _
    nRetVal)

```

```

bRet = swModel.EditRebuild3

```

```

End Sub

```

```

Public swApp As SldWorks

```

```

End Class

```

Edit Mate Reference Example (C#)

This example shows how to insert and edit a mate reference.

```
//-----
// Preconditions:
// 1. Verify that the specified part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Adds a mate reference.
// 3. Edits the mate reference.
// 4. Expand MateReferences in the FeatureManager design tree
//    and click MyDefault<1>.
// 5. Examine the graphics area and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Entity swPlane = default(Entity);
            Face2 swFace = default(Face2);
            Entity swFaceEntity = default(Entity);
            MateReference swMateReference = default(MateReference);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Insert mate reference
            status = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, true, 1, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swPlane = (Entity)swSelectionMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID2("", "FACE", 0.00835786916030656, 0.00429540237419701, 0, true, 2, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.InsertMateReference2("Default", null, 0, 1, false, null, 0, 2, false, null,
0, 0);
            swModel.ClearSelection2(true);

            //Edit mate reference
            status = swModelDocExt.SelectByID2("", "FACE", 0.0117890806857872, 0.00419179196288511, 0.00999999999999991, false, 0, null, 0);
            swFace = (Face2)swSelectionMgr.GetSelectedObject6(1, -1);
            swFaceEntity = (Entity)swFace;
            status = swModelDocExt.SelectByID2("Default<1>", "POSGROUP", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swMateReference = (MateReference)swFeature.GetSpecificFeature2();
            swModel.ClearSelection2(true);
            status = swMateReference.Edit("MyDefault", swPlane, (int)swMateReferenceType_e.swMateReferenceType_default, (int)swMateReferenceAlignment_e.sw
MateReferenceAlignment_Any, swFaceEntity, (int)swMateReferenceType_e.swMateReferenceType_default, (int)swMateReferenceAlignment_e.swMateReferenceAlignment
_Any, null, (int)swMateReferenceType_e.swMateReferenceType_default, (int)swMateReferenceAlignment_e.swMateReferenceAlignment_Any);
            Debug.Print("Mate reference modified and replaced? " + status);

            swModel.EditRebuild3();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Edit Mate Reference Example (VBA)

This example shows how to insert and edit a mate reference.

```
'-----
' Preconditions:
' 1. Verify that the specified part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Adds a mate reference.
' 3. Edits the mate reference.
' 4. Expand MateReferences in the FeatureManager design tree
'    and click MyDefault<1>.
' 5. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swPlane As SldWorks.Entity
Dim swFace As SldWorks.Face2
Dim swMateReference As SldWorks.MateReference
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Insert mate reference
    status = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swPlane = swSelectionMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("", "FACE", 8.35786916030656E-03, 4.29540237419701E-03, 0, True, 2, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.InsertMateReference2("Default", Nothing, 0, 1, False, Nothing, 0, 2, False, Nothing, 0, 0)
    swModel.ClearSelection2 True

    'Edit mate reference
    status = swModelDocExt.SelectByID2("", "FACE", 1.17890806857872E-02, 4.19179196288511E-03, 9.9999999999091E-03, False, 0, Nothing, 0)
    Set swFace = swSelectionMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("Default-<1>", "POSGROUP", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swMateReference = swFeature.GetSpecificFeature2
    swModel.ClearSelection2 True
    status = swMateReference.Edit("MyDefault", swPlane, swMateReferenceType_default, swMateReferenceAlignment_Any, swFace, swMateReferenceType_default,
    swMateReferenceAlignment_Any, Nothing, swMateReferenceType_default, swMateReferenceAlignment_Any)
    Debug.Print "Mate reference modified and replaced? " & status

    swModel.EditRebuild3

End Sub
```

Edit Mate Reference Example (VB.NET)

This example shows how to insert and edit a mate reference.

```

-----
Preconditions:
1. Verify that the specified part to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the part.
2. Adds a mate reference.
3. Edits the mate reference.
4. Expand MateReferences in the FeatureManager design tree
   and click MyDefault<1>.
5. Examine the graphics area and Immediate window.
NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swFeatureMgr As FeatureManager
        Dim swSelectionMgr As SelectionMgr
        Dim swPlane As Entity
        Dim swFace As Face2
        Dim swMateReference As MateReference
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Insert mate reference
        status = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swPlane = swSelectionMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID2("", "FACE", 0.00835786916030656, 0.00429540237419701, 0, True, 2, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.InsertMateReference2("Default", Nothing, 0, 1, False, Nothing, 0, 2, False, Nothing, 0, 0)
        swModel.ClearSelection2(True)

        'Edit mate reference
        status = swModelDocExt.SelectByID2("", "FACE", 0.0117890806857872, 0.00419179196288511, 0.00999999999999991, False, 0, Nothing, 0)
        swFace = swSelectionMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID2("Default-<1>", "POSGROUP", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swMateReference = swFeature.GetSpecificFeature2
        swModel.ClearSelection2(True)
        status = swMateReference.Edit("MyDefault", swPlane, swMateReferenceType_e.swMateReferenceType_default, swMateReferenceAlignment_e.swMateReferenceAlignment_Any, swFace, swMateReferenceType_e.swMateReferenceType_default, swMateReferenceAlignment_e.swMateReferenceAlignment_Any, Nothing, swMateReferenceType_e.swMateReferenceType_default, swMateReferenceAlignment_e.swMateReferenceAlignment_Any)
        Debug.Print("Mate reference modified and replaced? " & status)

        swModel.EditRebuild3()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Set Radial Dimension Leader Example (C#)

This example shows how to attach a radial dimension leader to an arc extension line.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Edits the sketch and creates a fillet.
// 3. Attaches the radial dimension leader to the arc extension
//    line.
// 4. Gets whether the radial dimension leader is attached to
//    the arc extension line.
// 5. Examine the graphics area, then press F5.
// 6. Exits the sketch.
// 7. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open the part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Edit the sketch and create a fillet
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.EditSketch();
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Point1", "SKETCHPOINT", -0.0811067833265636, 0.0389478433654258, 0, false, 0, null, 0);
            swSketchManager = swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateFillet(0.01, (int)swConstrainedCornerAction_e.swConstrainedCornerDeleteGeometry);

            //Select and set the radial dimension
            status = swModelDocExt.SelectByID2("D1@Sketch1@box.SLDPRT", "DIMENSION", -5.09218235791179E-02, 2.23786104078373E-02, 6.93106363229314E-03,
false, 0, null, 0);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swDisplayDimension = (DisplayDimension)swSelectionManager.GetSelectedObject6(1, -1);
            swDisplayDimension.ArcExtensionLineOrOppositeSide = true;
            Debug.Print("Leader attached to arc extension line? " + swDisplayDimension.ArcExtensionLineOrOppositeSide);

            System.Diagnostics.Debugger.Break();
            //Examine the graphics area, then press F5

            //Exit the sketch
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Radial Dimension Leader Example (VBA)

This example shows how to attach a radial dimension leader to an arc extension line.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Edits the sketch and creates a fillet.
' 3. Attaches the radial dimension leader to the arc extension
'    leader.
' 4. Gets whether the radial dimension leader is attached to
'    the arc extension leader.
' 5. Examine the graphics area, then press F5.
' 6. Exits the sketch.
' 7. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open the part
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Edit the sketch and create a fillet
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditSketch
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Point1", "SKETCHPOINT", -8.11067833265636E-02, 3.89478433654258E-02, 0, False, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateFillet(0.01, swConstrainedCornerAction_e.swConstrainedCornerDeleteGeometry)

    'Select and set the radial dimension
    status = swModelDocExt.SelectByID2("D1@Sketch1@box.SLDPR", "DIMENSION", -5.09218235791179E-02, 2.23786104078373E-02, 6.93106363229314E-03, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)
    swDisplayDimension.ArcExtensionLineOrOppositeSide = True
    Debug.Print "Leader attached to arc extension line? " & swDisplayDimension.ArcExtensionLineOrOppositeSide

    Stop
    'Examine the graphics area, then press F5

    'Exit the sketch
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

End Sub
```

Set Radial Dimension Leader Example (VB.NET)

This example shows how to attach a radial dimension leader to an arc extension line.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Opens the part.
' 2. Edits the sketch and creates a fillet.
' 3. Attaches the radial dimension leader to the arc extension
'    leader.
' 4. Gets whether the radial dimension leader is attached to
'    the arc extension leader.
' 5. Examine the graphics area, then press F5.
' 6. Exits the sketch.
' 7. Examine the Immediate window.
'-----
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swSelectionMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open the part
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Edit the sketch and create a fillet
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Point1", "SKETCHPOINT", -0.0811067833265636, 0.0389478433654258, 0, False, 0, Nothing, 0)
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateFillet(0.01, swConstrainedCornerAction_e.swConstrainedCornerDeleteGeometry)

        'Select and set the radial dimension
        status = swModelDocExt.SelectByID2("D1@Sketch1@box.SLDPRT", "DIMENSION", -5.09218235791179E-02, 2.23786104078373E-02, 6.93106363229314E-03, False,
0, Nothing, 0)
        Set swSelectionMgr = swModel.SelectionManager
        Set swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)
        swDisplayDimension.ArcExtensionLineOrOppositeSide = True
        Debug.Print "Leader attached to arc extension line? " & swDisplayDimension.ArcExtensionLineOrOppositeSide

        Stop
        'Examine the graphics area, then press F5

        'Exit the sketch
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Edit Radial Explode Step Example (VBA)

This example shows how to edit a radial explode step in an explode view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\RadialExplodeStep.sldasm.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates and explode view and gets the first explode step.
' 2. Gets the components to move.
' 3. Prints the current properties of the explode step.
' 4. Modifies the explode step properties.
' 5. Re-sets the components to move.
' 6. Re-sets the explode and diverge direction entities.
' 7. Inspect the Immediate window and the graphics area.
'
' Note: Because the model is used elsewhere, do not save any changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.AssemblyDoc
Dim swMdl As SldWorks.ModelDoc2
Dim explStep As SldWorks.ExplodeStep
Dim config As SldWorks.Configuration
Dim comp As SldWorks.Component2
Dim var As Variant
Dim i As Long
Dim transDir As SldWorks.Feature
Dim divergeDir As SldWorks.Feature
Dim num As Double
Dim selComp(2) As SldWorks.Component2
Dim sel3 As SldWorks.Feature
Dim sel4 As SldWorks.Feature
Dim transDir1 As SldWorks.Edge
Dim divergeDir1 As SldWorks.Feature
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set config = swModel.ConfigurationManager.ActiveConfiguration
    Set swMdl = swModel

    Call swModel.AutoExplode

    Set explStep = config.GetExplodeStep(0)

    Debug.Print "Pre-Modification"
    Debug.Print ""

    var = explStep.GetComponents()
    Debug.Print "Components to move:"
    For i = 0 To UBound(var)
        Set comp = var(i)
        Debug.Print " " & comp.Name2
    Next

    Debug.Print "Diverge from axis? " & explStep.DivergeFromAxis
    Debug.Print "Explode step: " & explStep.Name
    Debug.Print "Explode distance (m): " & explStep.ExplodeDistance
    Debug.Print "Explode step type as defined in swAssemblyExplodeStepType_e: " & explStep.ExplodeStepType
    Debug.Print "Rotation angle (rad): " & explStep.RotationAngle
    Debug.Print "Reverse rotation direction? " & explStep.ReverseRotationDirection
    Debug.Print "Reverse translation direction? " & explStep.ReverseTranslationDirection
    Debug.Print "Rotate about each component's origin? " & explStep.RotateAboutEachComponentOrigin

    Set divergeDir = explStep.DivergeDirection

    swMdl.ClearSelection2 (True)

    num = 3.1415 / 4

    explStep.DivergeFromAxis = Not explStep.DivergeFromAxis
    explStep.ExplodeDistance = 0.2
    explStep.RotationAngle = num
    explStep.ReverseRotationDirection = Not explStep.ReverseRotationDirection
    explStep.ReverseTranslationDirection = Not explStep.ReverseTranslationDirection
    explStep.RotateAboutEachComponentOrigin = Not explStep.RotateAboutEachComponentOrigin

    swMdl.ClearSelection2 (True)

```

```
boolstatus = swMdl.Extension.SelectByID2("Part2-1@RadialExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
boolstatus = swMdl.Extension.SelectByID2("Part2-2@RadialExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
boolstatus = swMdl.Extension.SelectByID2("Part2-3@RadialExplodeStep", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
```

```
Set selComp(0) = swMdl.SelectionManager.GetSelectedObject6(1, -1)
Set selComp(1) = swMdl.SelectionManager.GetSelectedObject6(2, -1)
Set selComp(2) = swMdl.SelectionManager.GetSelectedObject6(3, -1)
```

```
Call explStep.SetComponents(selComp)
```

```
boolstatus = swMdl.Extension.SelectByID2("Axis5", "AXIS", 0, 0, 0, True, 2 + 32, Nothing, 0) ' Explode direction entity
boolstatus = swMdl.Extension.SelectByID2("Axis4", "AXIS", 0, 0, 0, True, 64, Nothing, 0) ' Diverge direction entity
```

```
Set sel3 = swMdl.SelectionManager.GetSelectedObject6(1, -1)
Set sel4 = swMdl.SelectionManager.GetSelectedObject6(2, -1)
```

```
explStep.SetExplodeDirection sel3, -1
explStep.DivergeDirection = sel4
```

```
Debug.Print ""
Debug.Print "Post-Modification"
Debug.Print ""
```

```
var = explStep.GetComponents()
Debug.Print "Components to move:"
For i = 0 To UBound(var)
    Set comp = var(i)
    Debug.Print " " & comp.Name2
Next
```

```
Debug.Print "Diverge from axis? " & explStep.DivergeFromAxis
Debug.Print "Explode step: " & explStep.Name
Debug.Print "Explode distance (m): " & explStep.ExplodeDistance
Debug.Print "Explode step type as defined in swAssemblyExplodeStepType_e: " & explStep.ExplodeStepType
Debug.Print "Rotation angle (rad): " & explStep.RotationAngle
Debug.Print "Reverse rotation direction? " & explStep.ReverseRotationDirection
Debug.Print "Reverse translation direction? " & explStep.ReverseTranslationDirection
Debug.Print "Rotate about each component's origin? " & explStep.RotateAboutEachComponentOrigin
```

```
Set divergeDir = explStep.DivergeDirection
```

```
End Sub
```

Edit Spline Example (VBA)

This example shows how to modify the control points and knots of a spline.

```

'-----
' Preconditions:
' 1. Ensure the specified template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a spline sketch.
' 2. Gets its B-curve parameters.
' 3. Modifies a control point in the spline.
' 4. Modifies a knot in the spline.
' 5. When the macro stops, select the spline in the graphics area and press F5.
' 6. Gets the edited B-curve parameters.
' 7. Inspect the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim swPart As PartDoc
Dim skSegment As SldWorks.SketchSegment
Dim swSkSpl As SldWorks.SketchSpline
Dim swSketch As SldWorks.Sketch
Dim swSkMan As SldWorks.SketchManager
Dim swSplineParamData As SldWorks.SplineParamData
Dim swCurve As SldWorks.Curve
Dim boolstatus As Boolean
Dim vSketchSeg As Variant
Dim i As Integer, j As Integer, k As Integer
Dim swSheetWidth As Double
Dim swSheetHeight As Double
Dim pointArray As Variant
Dim points() As Double
Dim varCtrlPoints As Variant
Dim varKnotPoints As Variant
Dim weights As Variant

```

Sub main()

```

    Set swApp = Application.SldWorks
    swSheetWidth = 0
    swSheetHeight = 0
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Part.prt", 0, swSheetWidth, swSheetHeight)

```

```

    Set swPart = Part
    Set Part = swApp.ActiveDoc

```

```

    ReDim points(0 To 17) As Double
    points(0) = -0.112591397899791
    points(1) = -4.57086601325614E-02
    points(2) = 0
    points(3) = -7.86518975119748E-02
    points(4) = 2.83840071788291E-02
    points(5) = 0
    points(6) = -2.75869611486428E-02
    points(7) = 0
    points(8) = 0
    points(9) = 6.83154803141974E-02
    points(10) = 3.93969983009583E-02
    points(11) = 0
    points(12) = 0.10350046695487
    points(13) = 0.105807158470839
    points(14) = 0
    points(15) = 0.131212536078806
    points(16) = 0.121288640266982
    points(17) = 0
    pointArray = points

```

```

    Set skSegment = Part.SketchManager.CreateSpline((pointArray))
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

```

```

    Set swSkMan = Part.SketchManager
    Set swSketch = skSegment.GetSketch
    vSketchSeg = swSketch.GetSketchSegments

```

```

    Part.ClearSelection2 True

```

```

    For i = 0 To UBound(vSketchSeg)

```

```

        Set skSegment = vSketchSeg(i)
    
```

```

Set swCurve = skSegment.GetCurve
Set swSkSpl = skSegment

Debug.Print ""
Debug.Print "B-spline"
Debug.Print "Is a rational curve? " & swSkSpl.IsRationalCurve

If swSkSpl.IsRationalCurve Then
    weights = swSkSpl.GetControlVertexWeights
    For k = 0 To UBound(weights)
        Debug.Print weights(k)
    Next
End If

Set swSplineParamData = swCurve.GetBCurveParams5(False, False, False, False)

Debug.Print "The dimension is: " & swSplineParamData.Dimension
Debug.Print "The order is: " & swSplineParamData.Order
Debug.Print "The periodicity is: " & swSplineParamData.Periodic
Debug.Print "The control point count is: " & swSplineParamData.ControlPointsCount

Debug.Print "The knot point count is: " & swSplineParamData.KnotPointsCount
boolstatus = swSplineParamData.GetControlPoints(varCtrlPoints)
Debug.Print "Control Points:"
For j = 0 To UBound(varCtrlPoints)
    Debug.Print varCtrlPoints(j)
Next j

Debug.Print "The knot point count is: " & swSplineParamData.KnotPointsCount
boolstatus = swSplineParamData.GetKnotPoints(varKnotPoints)
Debug.Print "Knot Points:"
For j = 0 To UBound(varKnotPoints)
    Debug.Print varKnotPoints(j)
Next j

' Modify a control point
boolstatus = swSkSpl.ModifyControlPoint(3, -9.06980622069201E-02, 5.69034826745842E-02, 0#)
Debug.Print "Modified control point #3."
Part.ForceRebuild3 True

' Modify a knot
boolstatus = swSkSpl.ModifyKnot(6, 0.41)
Debug.Print "Modified knot #6."
Part.ForceRebuild3 True

swSkMan.InsertSketch True

Next i
Debug.Print "Select the sketch in the graphics area and press F5"
Stop ' Select the sketch in the graphics area

Set swSketch = swSkMan.ActiveSketch
vSketchSeg = swSketch.GetSketchSegments

If (IsEmpty(vSketchSeg)) Then
    Debug.Print "Periodic B-spline not created"
Else

    Part.ClearSelection2 True

    For i = 0 To UBound(vSketchSeg)

        Set skSegment = vSketchSeg(i)
        Set swCurve = skSegment.GetCurve

        Debug.Print ""
        Debug.Print "Edited B-spline"
        Debug.Print ""

        Set swSplineParamData = swCurve.GetBCurveParams5(False, False, False, False)

        Debug.Print "The dimension is: " & swSplineParamData.Dimension
        Debug.Print "The order is: " & swSplineParamData.Order
        Debug.Print "The periodicity is: " & swSplineParamData.Periodic
        Debug.Print "The control point count is: " & swSplineParamData.ControlPointsCount

        Debug.Print "The knot point count is: " & swSplineParamData.KnotPointsCount
        boolstatus = swSplineParamData.GetControlPoints(varCtrlPoints)
        Debug.Print "Control Points:"
        For j = 0 To UBound(varCtrlPoints)
            Debug.Print varCtrlPoints(j)
        Next j
    Next i

```

```
Debug.Print "The knot point count is: " & swSplineParamData.KnotPointsCount
boolstatus = swSplineParamData.GetKnotPoints(varKnotPoints)
Debug.Print "Knot Points:"
For j = 0 To UBound(varKnotPoints)
    Debug.Print varKnotPoints(j)
Next j

Next i

End If

End Sub
```

Enable Contour Selection Example (C#)

This example shows how to select the contour of a sketch region and extrude the selected region.

```
//-----
// Preconditions: Ensure that the specified document template exists.
//
// Postconditions: The selected sketch region is extruded.
// -----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace EnableContourSelection_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        object vSkLines;
        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2014\\templates\\Part.prtdot", 0, 0,
0);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = swModel.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDisplayOrigins, true);
            swModel.ClearSelection2(true);

            vSkLines = swModel.SketchManager.CreateCornerRectangle(-0.0390769010920735, 0.0405984975017191, 0, 0.0129020232818107,
-0.0166534302871355, 0);
            swModel.ClearSelection2(true);
            vSkLines = swModel.SketchManager.CreateCornerRectangle(-0.00751826843645631, 0.0156092594749566, 0, 0.0487922329685375,
-0.041704950991857, 0);
            swModel.ClearSelection2(true);
            swModel.SketchManager.InsertSketch(true);

            // Enable contour selection
            ((SelectionMgr)(swModel.SelectionManager)).EnableContourSelection = true;
            // Select a contour to extrude
            swModel.Extension.SelectByID2("Sketch1", "SKETCHREGION", 0, 0.01, 0, true, 4, null, 0);
            swModel.FeatureManager.FeatureExtrusion3(true, false, false, 0, 0, 0.01, 0.01, false, false, false,
false, 0, 0, false, false, false, false, true, true, true,
0, 0, false);
            // Disable contour selection
            ((SelectionMgr)(swModel.SelectionManager)).EnableContourSelection = false;
            swModel.ClearSelection2(true);

        }

        public SldWorks swApp;
    }
}
```

Enable Contour Selection Example (VBA)

This example shows how to select the contour of a sketch region and extrude the selected region.

```
'-----
' Preconditions: Ensure that the specified document template exists.
'
' Postconditions: The selected sketch region is extruded.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim vSkLines As Variant
Dim boolstatus As Boolean

Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2014\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swApp.ActiveDoc
    boolstatus = swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayOrigins, True)
    swModel.ClearSelection2 True

    vSkLines = swModel.SketchManager.CreateCornerRectangle(-3.90769010920735E-02, 4.05984975017191E-02, 0, 1.29020232818107E-02,
-1.66534302871355E-02, 0)
    swModel.ClearSelection2 True
    vSkLines = swModel.SketchManager.CreateCornerRectangle(-7.51826843645631E-03, 1.56092594749566E-02, 0, 4.87922329685375E-02,
-0.041704950991857, 0)
    swModel.ClearSelection2 True
    swModel.SketchManager.InsertSketch True

    ' Enable contour selection
    swModel.SelectionManager.EnableContourSelection = True
    ' Select a contour to extrude
    swModel.Extension.SelectByID2 "Sketch1", "SKETCHREGION", 0, 0.01, 0, True, 4, Nothing, 0
    swModel.FeatureManager.FeatureExtrusion3 True, False, False, 0, 0, 0.01, 0.01, False, False, False, False, 0, 0, False, False,
False, False, True, True, True, 0, 0, False
    ' Disable contour selection
    swModel.SelectionManager.EnableContourSelection = False
    swModel.ClearSelection2 True
End Sub
```

Enable Contour Selection Example (VB.NET)

This example shows how to select the contour of a sketch region and extrude the selected region.

```
'-----
' Preconditions: Ensure that the specified document template exists.
' Postconditions: The selected sketch region is extruded.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim vSkLines As Object
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2014\templates\Part.prtdot", 0, 0, 0)
        swModel = swApp.ActiveDoc
        boolstatus = swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayOrigins, True)
        swModel.ClearSelection2(True)
```

```
        vSkLines = swModel.SketchManager.CreateCornerRectangle(-0.0390769010920735, 0.0405984975017191, 0, 0.0129020232818107,
-0.0166534302871355, 0)
        swModel.ClearSelection2(True)
        vSkLines = swModel.SketchManager.CreateCornerRectangle(-0.00751826843645631, 0.0156092594749566, 0, 0.0487922329685375,
-0.041704950991857, 0)
        swModel.ClearSelection2(True)
        swModel.SketchManager.InsertSketch(True)
```

```
        ' Enable contour selection
        swModel.SelectionManager.EnableContourSelection = True
        ' Select a contour to extrude
        swModel.Extension.SelectByID2("Sketch1", "SKETCHREGION", 0, 0.01, 0, True, 4, Nothing, 0)
        swModel.FeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.01, 0.01, False, False, False, False, 0, 0, False,
False, False, False, True, True, True, 0, 0, False)
        ' Disable contour selection
        swModel.SelectionManager.EnableContourSelection = False
        swModel.ClearSelection2(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Enumerate Bodies Example (C++ COM)

This example shows how to use IEnumBodies to traverse the list of body objects. The IBody objects returned in IEnumBodies are the bodies associated with reference surfaces. This list does not include the part body itself. IEnumBodies::Next allows your application to get each individual body and then call appropriate methods for that body.

SOLIDWORKS API represents each reference surface with two IBody objects, one for the front face and one for the back face. The faces for each body pair should have Normal vectors that are opposite.

NOTE: Replace *public_documents* with the path where the header files for the general helper functions are installed on your computer. This path is typically **C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 20nn**, where *nn* is the 2-digit release year of the version of SOLIDWORKS Design that you are running.

```
// Precompiled header
#include "stdafx.h"

#include <public_documents/appcomm/amapp.h>
#include "userapp.h"

void WalkGTBody()
{
    LPMODELDOC m_ModelDoc = NULL;
    HRESULT hres = NOERROR;

    hres = UserApp->getSWApp()->get_IActiveDoc( &m_ModelDoc ); // Retrieve IModelDoc pointer
    if( m_ModelDoc == NULL )
        return;

    LPPARTDOC m_PartDoc = NULL;
    hres = m_ModelDoc->QueryInterface( IID_IPartDoc, (LPVOID *)&m_PartDoc);

    walkBody(m_PartDoc);

    m_ModelDoc ->Release();
    m_PartDoc->Release();
}

void walkBody(LPPARTDOC pPartDoc)
{
    LPENUMBODIES pBodyEnum = NULL;
    HRESULT resultVal = pPartDoc->EnumRelatedBodies(&pBodyEnum);
    LPBODY pOldBody;
    resultVal = pBodyEnum->Next(1, &pOldBody, NULL); // Two bodies for each RefSurface
    while (S_OK == resultVal)
    {
        LPENUMFACES pFaceEnum = NULL;
        resultVal = pOldBody->EnumFaces(&pFaceEnum);
        int faceNum = 0;
        LPFACE pFace;
        resultVal = pFaceEnum->Next(1, &pFace, NULL);
        while (S_OK == resultVal)
        {
            VARIANT normal;
            faceNum++;

            LPBODY pCheckBody;
            resultVal = pFace->IGetBody(&pCheckBody);
            if (pCheckBody != NULL) pCheckBody->Release();

            pFace->Release();
            pFace = NULL;
            resultVal = pFaceEnum->Next(1, &pFace, NULL);
        }

        if (pFaceEnum != NULL) pFaceEnum->Release();

        pOldBody ->Release();
        pOldBody = NULL;
        resultVal = pBodyEnum->Next(1, &pOldBody, NULL);
    }

    if (pBodyEnum != NULL) pBodyEnum->Release();
}
```


Evaluate Curves Defined in Sketch Space Example (VBA)

This example shows how to evaluate curves that were defined in the space of a sketch.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a sketch, opens the sketch, and selects a curve
'    in that sketch.
' 3. Evaluates the curve.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
' Define two types
Type DoubleRec
    dValue As Double
End Type
```

```
Type Long2Rec
    iLower As Long
    iUpper As Long
End Type
```

```
' Extract two integer values from a single double value
' by assigning a DoubleRec to the double value and then
' copying the value to Long2Rec and
' extracting the integer values
```

Function ExtractFields(ByVal dValue As Double, iLower As Long, iUpper As Long)

```
    Dim dr                As DoubleRec
    Dim i2r               As Long2Rec
```

```
    ' Set the double value
    dr.dValue = dValue
```

```
    ' Copy the values
    LSet i2r = dr
```

```
    ' Extract the values
    iLower = i2r.iLower
    iUpper = i2r.iUpper
```

End Function

Sub ProcessCurve(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swCurve As SldWorks.Curve)

```
    Dim swMathUtil        As SldWorks.MathUtility
    Dim swXform           As SldWorks.MathTransform
    Dim nStartParam       As Double
    Dim nEndParam         As Double
    Dim bIsClosed         As Boolean
    Dim bIsPeriodic       As Boolean
    Dim vStartEval        As Variant
    Dim vEndEval          As Variant
    Dim nSuccessStart     As Long
    Dim nEndStart         As Long
    Dim nDummy            As Long
    Dim nStartPt(2)       As Double
    Dim vStartPt          As Variant
    Dim swStartPt         As SldWorks.MathPoint
    Dim nStartTanPt(2)    As Double
    Dim vStartTanPt       As Variant
    Dim swStartTanPt      As SldWorks.MathPoint
    Dim nEndPt(2)         As Double
    Dim vEndPt            As Variant
    Dim swEndPt           As SldWorks.MathPoint
    Dim nEndTanPt(2)      As Double
    Dim vEndTanPt         As Variant
    Dim swEndTanPt        As SldWorks.MathPoint
    Dim bRet              As Boolean
```

```
    Set swMathUtil = swApp.GetMathUtility
    Set swXform = swSketch.ModelToSketchTransform
    Set swXform = swXform.Inverse
    bRet = swCurve.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic): Debug.Assert bRet
    vStartEval = swCurve.Evaluate(nStartParam)
    vEndEval = swCurve.Evaluate(nEndParam)
```

```
    ExtractFields vStartEval(6), nSuccessStart, nDummy
    ExtractFields vEndEval(6), nEndStart, nDummy
    nStartPt(0) = vStartEval(0):      nStartPt(1) = vStartEval(1):      nStartPt(2) = vStartEval(2)
    vStartPt = nStartPt
    Set swStartPt = swMathUtil.CreatePoint((vStartPt))
    Set swStartPt = swStartPt.MultiplyTransform(swXform)
```

```
    nStartTanPt(0) = vStartEval(3):   nStartTanPt(1) = vStartEval(4):   nStartTanPt(2) = vStartEval(5)
    vStartTanPt = nStartTanPt
    Set swStartTanPt = swMathUtil.CreatePoint((vStartTanPt))
    Set swStartTanPt = swStartPt.MultiplyTransform(swXform)
```

```

nEndPt(0) = vEndEval(0):          nEndPt(1) = vEndEval(1):          nEndPt(2) = vEndEval(2)
vEndPt = nEndPt
Set swEndPt = swMathUtil.CreatePoint((vEndPt))
Set swEndPt = swEndPt.MultiplyTransform(swXform)

nEndTanPt(0) = vEndEval(3):        nEndTanPt(1) = vEndEval(4):        nEndTanPt(2) = vEndEval(5)
vEndTanPt = nEndTanPt
Set swEndTanPt = swMathUtil.CreatePoint((vEndTanPt))
Set swEndTanPt = swEndPt.MultiplyTransform(swXform)

Debug.Print "IsClosed      = " & bIsClosed
Debug.Print "IsPeriodic   = " & bIsPeriodic
Debug.Print ""

Debug.Print "Start (sketch)"
Debug.Print "  Point      = (" & vStartEval(0) * 1000# & ", " & vStartEval(1) * 1000# & ", " & vStartEval(2) * 1000# & ") mm"
Debug.Print "  Tangent    = (" & vStartEval(3) & ", " & vStartEval(4) & ", " & vStartEval(5) & ")"
Debug.Print "  Success     = " & nSuccessStart

Debug.Print "Finish (sketch)"

Debug.Print "  Point      = (" & vEndEval(0) * 1000# & ", " & vEndEval(1) * 1000# & ", " & vEndEval(2) * 1000# & ") mm"
Debug.Print "  Tangent    = (" & vEndEval(3) & ", " & vEndEval(4) & ", " & vEndEval(5) & ")"
Debug.Print "  Success     = " & nEndStart

Debug.Print "Start (model)"
mm" Debug.Print "  Point      = (" & swStartPt.ArrayData(0) * 1000# & ", " & swStartPt.ArrayData(1) * 1000# & ", " & swStartPt.ArrayData(2) * 1000# & ")
Debug.Print "  Tangent    = (" & swStartTanPt.ArrayData(0) & ", " & swStartTanPt.ArrayData(1) & ", " & swStartTanPt.ArrayData(2) & ")"
Debug.Print "Finish (model)"

Debug.Print "  Point      = (" & swEndPt.ArrayData(0) * 1000# & ", " & swEndPt.ArrayData(1) * 1000# & ", " & swEndPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "  Tangent    = (" & swEndTanPt.ArrayData(0) & ", " & swEndTanPt.ArrayData(1) & ", " & swEndTanPt.ArrayData(2) & ")"

End Sub

Sub main()

Dim swApp          As SldWorks.SldWorks
Dim swModel        As SldWorks.ModelDoc2
Dim swModelDocExt  As SldWorks.ModelDocExtension
Dim swSelMgr       As SldWorks.SelectionMgr
Dim swSkSeg        As SldWorks.SketchSegment
Dim swSketch       As SldWorks.Sketch
Dim swCurve        As SldWorks.Curve
Dim errors         As Long
Dim warnings       As Long
Dim fileName       As String
Dim status         As Boolean

Set swApp = Application.SldWorks
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
swModel.EditSketch
swModel.ClearSelection2 True
status = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0.142211893757474, 3.33699273467745E-02, -1.57877105588811E-02, False, 0, Nothing, 0)
Set swSelMgr = swModel.SelectionManager
Set swSkSeg = swSelMgr.GetSelectedObject6(1, -1)
Set swSketch = swSkSeg.GetSketch
Set swCurve = swSkSeg.GetCurve

ProcessCurve swApp, swModel, swSketch, swCurve

End Sub

```

Evaluate Points on Surface Example (VBA)

This example shows how to evaluate points on a surface.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Select a face.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Evaluates the selection point on the
'    selected face.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swSurf As SldWorks.Surface
    Dim vSelPt As Variant
    Dim vClosePt As Variant
    Dim vEval As Variant

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject5(1)
    Set swSurf = swFace.GetSurface
    vSelPt = swSelMgr.GetSelectionPoint(1)
    vClosePt = swSurf.GetClosestPointOn(vSelPt(0), vSelPt(1), vSelPt(2))
    vEval = swSurf.EvaluateAtPoint(vClosePt(0), vClosePt(1), vClosePt(2))
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Sel Pt      = (" & vSelPt(0) * 1000 & ", " & vSelPt(1) * 1000 & ", " & vSelPt(2) * 1000 & ") mm"
    Debug.Print " Close Pt      = (" & vClosePt(0) * 1000 & ", " & vClosePt(1) * 1000 & ", " & vClosePt(2) * 1000 & ") mm"
    Debug.Print " Close U-V      = [" & vClosePt(3) & ", " & vClosePt(4) & "]"
    Debug.Print " Normal         = (" & vEval(0) & ", " & vEval(1) & ", " & vEval(2) & ")"
    Debug.Print " Dirn 1         = (" & vEval(3) & ", " & vEval(4) & ", " & vEval(5) & ")"
    Debug.Print " Dirn 2         = (" & vEval(6) & ", " & vEval(7) & ", " & vEval(8) & ")"
    Debug.Print " Curvat 1       = " & vEval(9)
    Debug.Print " Curvat 2       = " & vEval(10)

End Sub
```

Exclude Faces Before Flattening Example (C#)

This example shows how to exclude faces from a Flat-Pattern feature.

```
// -----
// Preconditions: Verify that the specified part to open exists.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Excludes 22 faces from Flat-Pattern1.
// 3. Unsuppresses Flat-Pattern1.
// 4. Examine the graphics area and FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere,
// do not save changes.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Specialized;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace GetExcludedFaces_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            ModelView View = default(ModelView);
            bool boolstatus = false;
            int longstatus = 0;
            int longWarnings = 0;
            Feature swFeature = default(Feature);
            FlatPatternFeatureData swFlatPatternFeatureData = default(FlatPatternFeatureData);
            NameValueCollection PIDCollection = new NameValueCollection();

            if (swApp == null)
                return;

            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\FPF.SLDPRT",
1, 0, "", ref longstatus, ref longWarnings);
            if (Part == null) { ErrorMsg(swApp, "Failed to open FPF.SLDPRT"); }

            PIDCollection = PIDInitializer();

            boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.EditUnsuppress2();

            View = (ModelView)Part.ActiveView;
            double[] rect;
            rect = null;
            View.GraphicsRedraw(rect);

            boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.EditSuppress2();
            if (boolstatus == false) { ErrorMsg(swApp, "EditSuppress Failed"); }

            Part = (ModelDoc2)swApp.ActiveDoc;
            swFeature = (Feature)Part.FirstFeature();
            while ((swFeature != null))
            {
                if (swFeature.GetTypeName() == "FlatPattern")
                {
                    //Select excluded faces
                    swFlatPatternFeatureData = (FlatPatternFeatureData)swFeature.GetDefinition();

                    swFlatPatternFeatureData.AccessSelections(Part, null);

                    //Pick by PID
                    boolstatus = SelectByPID(swApp, Part, "selection1", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection2", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection3", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection4", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection5", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection6", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection7", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection8", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection9", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection10", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection11", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection12", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection13", PIDCollection, 1, true);
                    boolstatus = SelectByPID(swApp, Part, "selection14", PIDCollection, 1, true);
                }
            }
        }
    }
}
```

```

        boolstatus = SelectByPID(swApp, Part, "selection15", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection16", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection17", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection18", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection19", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection20", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection21", PIDCollection, 1, true);
        boolstatus = SelectByPID(swApp, Part, "selection22", PIDCollection, 1, true);

        Face2[] faces = new Face2[22];
        int j = 0;
        int count = ((SelectionMgr)Part.SelectionManager).GetSelectedObjectCount2(1);

        for (j = 0; j <= 21; j++)
        {
            faces[j] = (Face2)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(j+1, 1);
        }

        object facesVar = null;
        facesVar = faces;

        swFlatPatternFeatureData.ExcludedFaces = facesVar;
        boolstatus = swFeature.ModifyDefinition(swFlatPatternFeatureData, Part, null);
        if (boolstatus == false) { ErrorMessage(swApp, "Selecting excluded faces failed"); }

        boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
        boolstatus = Part.EditUnsuppress2();
        if (boolstatus == false) { ErrorMessage(swApp, "EditUnsuppress Failed"); }

        View.GraphicsRedraw(rect);
    }
    swFeature = (Feature)swFeature.GetNextFeature();
}

Part.ForceRebuild3(false);
}

public string ErrorMessage(SldWorks SwApp, string Message)
{
    SwApp.SendMsgToUser2(Message, 0, 0);
    SwApp.RecordLine("**** WARNING - General");
    SwApp.RecordLine("**** " + Message);
    SwApp.RecordLine("");
    return "";
}

public NameValueCollection PIDInitializer()
{
    NameValueCollection PIDCollection = new NameValueCollection();
    string selection1 = null;
    string selection2 = null;
    string selection3 = null;
    string selection4 = null;
    string selection5 = null;
    string selection6 = null;
    string selection7 = null;
    string selection8 = null;
    string selection9 = null;
    string selection10 = null;
    string selection11 = null;
    string selection12 = null;
    string selection13 = null;
    string selection14 = null;
    string selection15 = null;
    string selection16 = null;
    string selection17 = null;
    string selection18 = null;
    string selection19 = null;
    string selection20 = null;
    string selection21 = null;
    string selection22 = null;

    //Constants
    selection1 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,103,1,0,0,120,1,117,81,77,75,2,81,20,61,38,154,17,45,50,201,69,
selection1 = selection1 +
"6,107,203,193,165,35,32,19,229,54,88,142,152,156,65,145,15,214,70,150,211,49,205,224,215,184,36,253,148,180,194,63,18,188,212,82,34
selection1 = selection1 + ",Type=1";

    selection2 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,79,1,0,0,120,1,109,81,93,75,2,81,16,61,38,126,68,244,144,69,246
selection2 = selection2 +
"5,46,128,220,162,184,198,10,196,100,7,37,14,220,137,61,245,176,177,41,97,94,146,49,99,181,176,142,100,83,222,64,190,226,188,237,106
selection2 = selection2 + ",Type=1";

    selection3 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,37,1,0,0,120,1,101,80,203,74,195,80,16,61,53,168,21,113,225,3,
selection3 = selection3 +
"78,229,112,86,201,195,149,205,37,71,197,250,194,114,170,97,114,161,147,224,207,169,164,245,136,45,123,191,227,12,255,73,52,133,92,2
selection3 = selection3 + ",Type=1";

    selection4 =

```

```
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 78, 1, 0, 0, 120, 1, 109, 81, 193, 74, 2, 81, 20, 61, 38, 90, 17, 45, 178, 200, 246
selection4 = selection4 +
", 219, 34, 27, 217, 57, 144, 155, 111, 173, 176, 212, 105, 122, 80, 20, 63, 162, 96, 98, 143, 227, 154, 191, 110, 151, 140, 31, 147, 122, 64, 9, 241, 254, 175, 132, 96
selection4 = selection4 + ", Type=1";

selection5 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 80, 2, 0, 0, 95, 1, 0, 0, 120, 1, 109, 82, 77, 75, 2, 81, 20, 61, 102, 90, 17, 45, 50, 201, 69, 2
selection5 = selection5 +
"8, 215, 92, 171, 46, 187, 74, 78, 129, 212, 116, 123, 133, 109, 18, 147, 29, 228, 100, 31, 129, 55, 52, 7, 97, 217, 93, 95, 151, 208, 143, 73, 203, 82, 66, 118, 255, 91
selection5 = selection5 + ", Type=1";

selection6 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 89, 2, 0, 0, 97, 1, 0, 0, 120, 1, 133, 80, 203, 74, 195, 80, 16, 61, 181, 84, 43, 226, 194, 42,
selection6 = selection6 +
"235, 129, 218, 213, 119, 212, 45, 79, 219, 190, 252, 115, 151, 66, 82, 228, 100, 64, 139, 185, 217, 141, 21, 143, 178, 243, 9, 239, 205, 248, 111, 174, 61, 114, 213,
selection6 = selection6 + ", Type=1";

selection7 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 137, 2, 0, 0, 111, 1, 0, 0, 120, 1, 125, 82, 77, 75, 195, 64, 16, 125, 181, 180, 85, 196, 131,
selection7 = selection7 +
"8, 77, 139, 189, 182, 60, 107, 129, 7, 171, 137, 192, 158, 11, 235, 215, 157, 100, 162, 109, 92, 144, 215, 93, 241, 142, 229, 253, 4, 222, 68, 31, 135, 117, 55, 60, 11
selection7 = selection7 + ", Type=1";

selection8 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 80, 2, 0, 0, 97, 1, 0, 0, 120, 1, 101, 81, 77, 75, 2, 81, 20, 61, 38, 126, 68, 180, 200, 36, 23,
selection8 = selection8 +
"219, 202, 16, 147, 115, 32, 53, 207, 174, 57, 25, 162, 204, 32, 199, 7, 183, 167, 142, 249, 184, 49, 41, 81, 94, 83, 17, 48, 91, 84, 121, 170, 170, 202, 183, 102, 106,
selection8 = selection8 + ", Type=1";

selection9 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 77, 1, 0, 0, 120, 1, 109, 81, 77, 75, 2, 81, 20, 61, 54, 168, 69, 180, 200, 34, 215
selection9 = selection9 +
"146, 133, 25, 115, 32, 63, 223, 92, 229, 68, 73, 118, 80, 226, 131, 219, 179, 64, 221, 175, 109, 74, 152, 23, 100, 76, 217, 45, 172, 67, 217, 84, 48, 144, 175, 56, 51,
selection9 = selection9 + ", Type=1";

selection10 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 89, 2, 0, 0, 101, 1, 0, 0, 120, 1, 125, 82, 203, 74, 195, 64, 20, 61, 181, 214, 7, 34, 98, 21, 2
selection10 = selection10 +
"6, 163, 195, 118, 67, 207, 122, 164, 103, 13, 92, 189, 203, 243, 122, 160, 254, 236, 37, 55, 221, 96, 103, 64, 141, 177, 105, 190, 150, 122, 223, 58, 119, 79, 239, 26
selection10 = selection10 + ", Type=1";

selection11 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 191, 1, 0, 0, 37, 1, 0, 0, 120, 1, 93, 80, 203, 74, 195, 80, 16, 61, 53, 90, 21, 113, 225, 3, 25
selection11 = selection11 +
"2, 229, 218, 15, 215, 36, 39, 55, 144, 83, 13, 147, 115, 157, 4, 127, 78, 37, 165, 135, 44, 217, 101, 123, 125, 247, 255, 35, 150, 169, 255, 218, 55, 254, 120, 101, 23
selection11 = selection11 + ", Type=1";

selection12 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 77, 1, 0, 0, 120, 1, 109, 81, 209, 74, 2, 81, 16, 61, 102, 90, 17, 61, 100, 145, 18
selection12 = selection12 +
"5, 0, 185, 100, 123, 197, 137, 146, 236, 160, 40, 251, 8, 253, 169, 61, 137, 107, 222, 250, 186, 164, 252, 132, 101, 135, 148, 144, 221, 255, 149, 88, 124, 22, 37, 16
selection12 = selection12 + ", Type=1";

selection13 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 80, 2, 0, 0, 105, 1, 0, 0, 120, 1, 101, 82, 205, 74, 2, 81, 24, 61, 102, 90, 17, 45, 50, 201, 65
selection13 = selection13 +
"114, 14, 164, 230, 187, 43, 110, 155, 156, 220, 65, 142, 31, 172, 15, 108, 183, 105, 89, 147, 127, 215, 37, 242, 83, 202, 178, 180, 224, 1, 54, 44, 228, 29, 196, 34, 6
selection13 = selection13 + ", Type=1";

selection14 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 80, 1, 0, 0, 120, 1, 101, 81, 209, 74, 2, 65, 20, 61, 102, 90, 17, 61, 100, 145, 18
selection14 = selection14 +
", 114, 241, 230, 50, 38, 74, 50, 131, 2, 31, 220, 158, 26, 53, 92, 153, 148, 48, 47, 200, 8, 89, 45, 172, 67, 153, 148, 25, 200, 87, 156, 53, 204, 234, 84, 133, 91, 38, 1
selection14 = selection14 + ", Type=1";

selection15 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 191, 1, 0, 0, 38, 1, 0, 0, 120, 1, 93, 80, 203, 74, 195, 64, 20, 61, 53, 214, 7, 226, 194, 7, 25
selection15 = selection15 +
"40, 213, 165, 84, 225, 159, 179, 180, 203, 13, 28, 51, 117, 159, 169, 77, 185, 85, 242, 206, 196, 155, 22, 171, 36, 254, 99, 252, 212, 47, 249, 1, 100, 81, 101, 46, 0,
selection15 = selection15 + ", Type=1";

selection16 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 78, 1, 0, 0, 120, 1, 109, 81, 93, 75, 2, 65, 20, 61, 182, 104, 69, 244, 144, 69, 24
selection16 = selection16 +
", 153, 1, 217, 217, 214, 42, 182, 152, 52, 103, 144, 231, 135, 91, 83, 163, 134, 107, 147, 18, 230, 37, 171, 98, 86, 203, 164, 142, 100, 82, 102, 32, 171, 56, 175, 155
selection16 = selection16 + ", Type=1";

selection17 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 89, 2, 0, 0, 98, 1, 0, 0, 120, 1, 125, 81, 203, 74, 195, 64, 20, 61, 181, 214, 7, 226, 194, 42,
selection17 = selection17 +
", 148, 93, 15, 100, 87, 223, 145, 183, 60, 109, 248, 209, 159, 119, 201, 197, 27, 156, 12, 168, 179, 54, 237, 23, 164, 42, 178, 49, 223, 155, 249, 223, 92, 123, 228, 1
selection17 = selection17 + ", Type=1";

selection18 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 137, 2, 0, 0, 109, 1, 0, 0, 120, 1, 133, 81, 77, 75, 2, 81, 20, 61, 38, 90, 17, 45, 50, 201, 166
selection18 = selection18 +
", 46, 185, 166, 133, 110, 75, 222, 154, 231, 195, 170, 202, 183, 231, 202, 90, 155, 73, 38, 220, 194, 57, 121, 157, 21, 239, 72, 230, 227, 123, 99, 125, 20, 212, 220,
selection18 = selection18 + ", Type=1";

selection19 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 79, 1, 0, 0, 120, 1, 101, 81, 205, 74, 2, 81, 20, 254, 76, 212, 34, 90, 100, 145, 2
selection19 = selection19 +
"06, 128, 220, 108, 125, 25, 19, 38, 153, 65, 137, 31, 238, 141, 60, 211, 118, 221, 240, 207, 184, 4, 126, 64, 216, 46, 41, 248, 128, 127, 20, 133, 31, 138, 34, 41, 58,
selection19 = selection19 + ", Type=1";

selection20 =
```

```

"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 89, 2, 0, 0, 96, 1, 0, 0, 120, 1, 133, 80, 205, 74, 2, 81, 20, 254, 76, 52, 35, 90, 100, 161, 65
selection20 = selection20 +
"45, 64, 237, 234, 217, 234, 150, 167, 109, 47, 250, 115, 151, 92, 92, 224, 100, 64, 139, 181, 233, 141, 21, 14, 89, 145, 141, 121, 111, 230, 127, 99, 237, 19, 171, 46
selection20 = selection20 + ", Type=1";

selection21 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 191, 1, 0, 0, 38, 1, 0, 0, 120, 1, 93, 80, 203, 74, 195, 64, 20, 61, 53, 216, 86, 196, 133, 15,
selection21 = selection21 +
"87, 191, 92, 155, 156, 190, 129, 62, 213, 52, 189, 144, 105, 248, 231, 84, 186, 244, 136, 37, 123, 108, 39, 212, 246, 127, 68, 183, 17, 106, 252, 1, 7, 155, 101, 58, 6
selection21 = selection21 + ", Type=1";

selection22 =
"35, 29, 213, 113, 218, 129, 72, 162, 168, 88, 152, 178, 27, 137, 239, 153, 33, 2, 0, 0, 80, 1, 0, 0, 120, 1, 93, 81, 93, 75, 2, 81, 16, 61, 38, 126, 68, 244, 144, 69, 246,
selection22 = selection22 +
", 61, 3, 50, 179, 252, 10, 203, 17, 147, 29, 20, 249, 224, 238, 200, 53, 45, 199, 9, 54, 214, 37, 244, 99, 210, 14, 40, 33, 187, 223, 148, 16, 76, 36, 10, 148, 104, 71, 7
selection22 = selection22 + ", Type=1";

//Store constants in a collection
PIDCollection.Add("selection1", selection1);
PIDCollection.Add("selection2", selection2);
PIDCollection.Add("selection3", selection3);
PIDCollection.Add("selection4", selection4);
PIDCollection.Add("selection5", selection5);
PIDCollection.Add("selection6", selection6);
PIDCollection.Add("selection7", selection7);
PIDCollection.Add("selection8", selection8);
PIDCollection.Add("selection9", selection9);
PIDCollection.Add("selection10", selection10);
PIDCollection.Add("selection11", selection11);
PIDCollection.Add("selection12", selection12);
PIDCollection.Add("selection13", selection13);
PIDCollection.Add("selection14", selection14);
PIDCollection.Add("selection15", selection15);
PIDCollection.Add("selection16", selection16);
PIDCollection.Add("selection17", selection17);
PIDCollection.Add("selection18", selection18);
PIDCollection.Add("selection19", selection19);
PIDCollection.Add("selection20", selection20);
PIDCollection.Add("selection21", selection21);
PIDCollection.Add("selection22", selection22);

//Pass this back
return PIDCollection;
}

Append) public bool SelectByPID(SldWorks SwApp, ModelDoc2 Part, string PIDName, NameValueCollection PIDCollection, int Mark, bool
{
    object SelObj = null;
    byte[] PID = null;
    string[] PIDVariant = null;
    string PIDString = null;
    int EntityType = 0;
    int i = 0;
    bool boolstatus = false;

    //Get the string from the collection
    PIDString = "";
    PIDString = PIDCollection[PIDName];

    //Parse the string into an array
    char[] separator;
    separator = ",".ToCharArray();
    PIDVariant = PIDString.Split(separator);
    PID = new byte[PIDVariant.GetUpperBound(0) + 1];

    //Change to a byte array
    for (i = 0; i <= (PIDVariant.GetUpperBound(0) - 1); i++)
    {
        PID[i] = (Convert.ToByte(PIDVariant[i]));
    }

    SelObj = Part.Extension.GetObjectByPersistReference(PID);
    Face2 face = (Face2)SelObj;
    Entity entity = (Entity)face;
    SelectData data = ((SelectionMgr)Part.SelectionManager).CreateSelectData();
    data.Mark = 1;
    entity.Select4(true, data);

    Feature feat = (Feature)face.GetFeature();

    //Get the entity type
    string temp = (string)PIDVariant.GetValue(PIDVariant.GetUpperBound(0));
    EntityType = (int)Convert.ToDecimal(temp.Substring(temp.Length - 1));

    if (EntityType == 1)
        boolstatus = feat.Select2(true, 0);

    return boolstatus;
}

public SldWorks swApp;
}
}

```

Exclude Faces Before Flattening Example (VBA)

This example shows how to exclude faces from a Flat-Pattern feature.

```

'-----
' Preconditions: Verify that the specified part to open exists.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Excludes 22 faces from Flat-Pattern1.
' 3. Unsuppresses Flat-Pattern1.
' 4. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere,
' do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim SwApp                As SldWorks.SldWorks
    Dim Part                 As SldWorks.ModelDoc2
    Dim View                 As SldWorks.ModelView
    Dim boolstatus           As Boolean
    Dim longstatus           As Long
    Dim LongWarnings         As Long
    Dim swFeature            As SldWorks.Feature
    Dim swFlatPatternFeatureData As SldWorks.FlatPatternFeatureData
    Dim varFaces             As Variant
    Dim PIDCollection        As New Collection
    Dim intFeatureCount      As Integer

    Set SwApp = Application.SldWorks
    If SwApp Is Nothing Then Exit Sub

    Set Part = SwApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\FPF.SLDPRT", 1, 0, "",
longstatus, LongWarnings)
    If Part Is Nothing Then ErrorMessage SwApp, "Failed to open FPF.SLDPRT": GoTo LastLine

    Set PIDCollection = PIDInitializer

    'Traverse and Set excluded faces

    boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.EditUnsuppress2()

    Set View = Part.ActiveView
    Dim rect As Variant
    Set rect = Nothing
    View.GraphicsRedraw (rect)

    boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.EditSuppress2()
    If boolstatus = False Then ErrorMessage SwApp, "EditSuppress Failed": GoTo LastLine

    Set Part = SwApp.ActiveDoc
    Set swFeature = Part.FirstFeature
    While Not swFeature Is Nothing
        If swFeature.GetTypeName = "FlatPattern" Then
            'Select excluded faces
            Set swFlatPatternFeatureData = swFeature.GetDefinition

            swFlatPatternFeatureData.AccessSelections Part, Nothing

            'Pick by PID
            boolstatus = SelectByPID(SwApp, Part, "selection1", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection2", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection3", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection4", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection5", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection6", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection7", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection8", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection9", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection10", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection11", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection12", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection13", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection14", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection15", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection16", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection17", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection18", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection19", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection20", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection21", PIDCollection, 1, True)
            boolstatus = SelectByPID(SwApp, Part, "selection22", PIDCollection, 1, True)
        End If
    End While

```

```

Dim faces(0 To 21) As Object
Dim ii As Integer
For ii = 0 To 21
    Set faces(ii) = Part.SelectionManager.GetSelectedObject6(ii + 1, 1)
Next ii

Dim facesVar As Variant
facesVar = faces

swFlatPatternFeatureData.ExcludedFaces = facesVar
boolstatus = swFeature.ModifyDefinition(swFlatPatternFeatureData, Part, Nothing)
If boolstatus = False Then ErrorMessage SwApp, "Selecting excluded faces failed": GoTo LastLine

boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = Part.EditUnsuppress2()
If boolstatus = False Then ErrorMessage SwApp, "EditUnsuppress Failed": GoTo LastLine

View.GraphicsRedraw (rect)

End If
Set swFeature = swFeature.GetNextFeature
Wend

Part.ForceRebuild3 False

LastLine:
End Sub

Function ErrorMessage(SwApp As Object, Message As String)
    SwApp.SendMsgToUser2 Message, 0, 0
    SwApp.RecordLine "**** WARNING - General"
    SwApp.RecordLine "**** " & Message
    SwApp.RecordLine ""
End Function

Function PIDInitializer() As Collection

    Dim PIDCollection As New Collection
    Dim selection1 As String
    Dim selection2 As String
    Dim selection3 As String
    Dim selection4 As String
    Dim selection5 As String
    Dim selection6 As String
    Dim selection7 As String
    Dim selection8 As String
    Dim selection9 As String
    Dim selection10 As String
    Dim selection11 As String
    Dim selection12 As String
    Dim selection13 As String
    Dim selection14 As String
    Dim selection15 As String
    Dim selection16 As String
    Dim selection17 As String
    Dim selection18 As String
    Dim selection19 As String
    Dim selection20 As String
    Dim selection21 As String
    Dim selection22 As String

    'Constants
    selection1 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,103,1,0,0,120,1,117,81,77,75,2,81,20,61,38,154,17,45,50,201,69,
selection1 = selection1 & ",Type=1"

    selection2 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,79,1,0,0,120,1,109,81,93,75,2,81,16,61,38,126,68,244,144,69,246
selection2 = selection2 & ",Type=1"

    selection3 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,37,1,0,0,120,1,101,80,203,74,195,80,16,61,53,168,21,113,225,3,
selection3 = selection3 & ",Type=1"

    selection4 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,78,1,0,0,120,1,109,81,193,74,2,81,20,61,38,90,17,45,178,200,246
selection4 = selection4 & ",Type=1"

    selection5 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,95,1,0,0,120,1,109,82,77,75,2,81,20,61,102,90,17,45,50,201,69,2
selection5 = selection5 & ",Type=1"

    selection6 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,97,1,0,0,120,1,133,80,203,74,195,80,16,61,181,84,43,226,194,42,

```

```

selection6 = selection6 &
"235,129,218,213,119,212,45,79,219,190,252,115,151,66,82,228,100,64,139,185,217,141,21,143,178,243,9,239,205,248,111,174,61,114,213,
selection6 = selection6 & ",Type=1"

selection7 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,137,2,0,0,111,1,0,0,120,1,125,82,77,75,195,64,16,125,181,180,85,196,131,
selection7 = selection7 &
"8,77,139,189,182,60,107,129,7,171,137,192,158,11,235,215,157,100,162,109,92,144,215,93,241,142,229,253,4,222,68,31,135,117,55,60,11
selection7 = selection7 & ",Type=1"

selection8 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,97,1,0,0,120,1,101,81,77,75,2,81,20,61,38,126,68,180,200,36,23,
selection8 = selection8 &
"219,202,16,147,115,32,53,207,174,57,25,162,204,32,199,7,183,167,142,249,184,49,41,81,94,83,17,48,91,84,121,170,170,202,183,102,106,
selection8 = selection8 & ",Type=1"

selection9 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,77,1,0,0,120,1,109,81,77,75,2,81,20,61,54,168,69,180,200,34,215,
selection9 = selection9 &
"146,133,25,115,32,63,223,92,229,68,73,118,80,226,131,219,179,64,221,175,109,74,152,23,100,76,217,45,172,67,217,84,48,144,175,56,51,
selection9 = selection9 & ",Type=1"

selection10 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,101,1,0,0,120,1,125,82,203,74,195,64,20,61,181,214,7,34,98,21,2,
selection10 = selection10 &
"6,163,195,118,67,207,122,164,103,13,92,189,203,243,122,160,254,236,37,55,221,96,103,64,141,177,105,190,150,122,223,58,119,79,239,26
selection10 = selection10 & ",Type=1"

selection11 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,37,1,0,0,120,1,93,80,203,74,195,80,16,61,53,90,21,113,225,3,2,
selection11 = selection11 &
"2,229,218,15,215,36,39,55,144,83,13,147,115,157,4,127,78,37,165,135,44,217,101,123,125,247,255,35,150,169,255,218,55,254,120,101,23
selection11 = selection11 & ",Type=1"

selection12 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,77,1,0,0,120,1,109,81,209,74,2,81,16,61,102,90,17,61,100,145,1,
selection12 = selection12 &
"5,0,185,100,123,197,137,146,236,160,40,251,8,253,169,61,137,107,222,250,186,164,252,132,101,135,148,144,221,255,149,88,124,22,37,1,
selection12 = selection12 & ",Type=1"

selection13 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,105,1,0,0,120,1,101,82,205,74,2,81,24,61,102,90,17,45,50,201,6,
selection13 = selection13 &
"114,14,164,230,187,43,110,155,156,220,65,142,31,172,15,108,183,105,89,147,127,215,37,242,83,202,178,180,224,1,54,44,228,29,196,34,6
selection13 = selection13 & ",Type=1"

selection14 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,80,1,0,0,120,1,101,81,209,74,2,65,20,61,102,90,17,61,100,145,1,
selection14 = selection14 &
",114,241,230,50,38,74,50,131,2,31,220,158,26,53,92,153,148,48,47,200,8,89,45,172,67,153,148,25,200,87,156,53,204,234,84,133,91,38,1
selection14 = selection14 & ",Type=1"

selection15 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,38,1,0,0,120,1,93,80,203,74,195,64,20,61,53,214,7,226,194,7,2,
selection15 = selection15 &
"40,213,165,84,225,159,179,180,203,13,28,51,117,159,169,77,185,85,242,206,196,155,22,171,36,254,99,252,212,47,249,1,100,81,101,46,0,
selection15 = selection15 & ",Type=1"

selection16 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,78,1,0,0,120,1,109,81,93,75,2,65,20,61,182,104,69,244,144,69,2,
selection16 = selection16 &
",153,1,217,217,214,42,182,152,52,103,144,231,135,91,83,163,134,107,147,18,230,37,171,98,86,203,164,142,100,82,102,32,171,56,175,15,
selection16 = selection16 & ",Type=1"

selection17 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,98,1,0,0,120,1,125,81,203,74,195,64,20,61,181,214,7,226,194,42,
selection17 = selection17 &
",148,93,15,100,87,223,145,183,60,109,248,209,159,119,201,197,27,156,12,168,179,54,237,23,164,42,178,49,223,155,249,223,92,123,228,1
selection17 = selection17 & ",Type=1"

selection18 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,137,2,0,0,109,1,0,0,120,1,133,81,77,75,2,81,20,61,38,90,17,45,50,201,16,
selection18 = selection18 &
",46,185,166,133,110,75,222,154,231,195,170,202,183,231,202,90,155,73,38,220,194,57,121,157,21,239,72,230,227,123,99,125,20,212,220,
selection18 = selection18 & ",Type=1"

selection19 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,79,1,0,0,120,1,101,81,205,74,2,81,20,254,76,212,34,90,100,145,2,
selection19 = selection19 &
"06,128,220,108,125,25,19,38,153,65,137,31,238,141,60,211,118,221,240,207,184,4,126,64,216,46,41,248,128,127,20,133,31,138,34,41,58,
selection19 = selection19 & ",Type=1"

selection20 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,96,1,0,0,120,1,133,80,205,74,2,81,20,254,76,52,35,90,100,161,6,
selection20 = selection20 &
"45,64,237,234,217,234,150,167,109,47,250,115,151,92,92,224,100,64,139,181,233,141,21,14,89,145,141,121,111,230,127,99,237,19,171,4,
selection20 = selection20 & ",Type=1"

selection21 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,38,1,0,0,120,1,93,80,203,74,195,64,20,61,53,216,86,196,133,15,
selection21 = selection21 &
"87,191,92,155,156,190,129,62,213,52,189,144,105,248,231,84,186,244,136,37,123,108,39,212,246,127,68,183,17,106,252,1,7,155,101,58,6
selection21 = selection21 & ",Type=1"

selection22 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,80,1,0,0,120,1,93,81,93,75,2,81,16,61,38,126,68,244,144,69,246,
selection22 = selection22 &
",61,3,50,179,252,10,203,17,147,29,20,249,224,238,200,53,45,199,9,54,214,37,244,99,210,14,40,33,187,223,148,16,76,36,10,148,104,71,7
selection22 = selection22 & ",Type=1"

```

```

'Store constants in a collection.
PIDCollection.Add selection1, "selection1"

```

```

PIDCollection.Add selection2, "selection2"
PIDCollection.Add selection3, "selection3"
PIDCollection.Add selection4, "selection4"
PIDCollection.Add selection5, "selection5"
PIDCollection.Add selection6, "selection6"
PIDCollection.Add selection7, "selection7"
PIDCollection.Add selection8, "selection8"
PIDCollection.Add selection9, "selection9"
PIDCollection.Add selection10, "selection10"
PIDCollection.Add selection11, "selection11"
PIDCollection.Add selection12, "selection12"
PIDCollection.Add selection13, "selection13"
PIDCollection.Add selection14, "selection14"
PIDCollection.Add selection15, "selection15"
PIDCollection.Add selection16, "selection16"
PIDCollection.Add selection17, "selection17"
PIDCollection.Add selection18, "selection18"
PIDCollection.Add selection19, "selection19"
PIDCollection.Add selection20, "selection20"
PIDCollection.Add selection21, "selection21"
PIDCollection.Add selection22, "selection22"

```

```

'Pass this back
Set PIDInitializer = PIDCollection

```

End Function

```

Function SelectByPID(SwApp As SldWorks.SldWorks, Part As SldWorks.ModelDoc2, PIDName As String, PIDCollection As Collection,
Optional Mark As Long = 0, Optional Append As Boolean = False) As Boolean

```

```

Dim SelObj As Object
Dim PID() As Byte
Dim PIDVariant As Variant
Dim PIDString As String
Dim EntityType As Long
Dim i As Integer
Dim boolstatus As Boolean

```

```

'Get the string from the collection
PIDString = ""
PIDString = PIDCollection.Item(PIDName)

```

```

'Parse the string into an array
PIDVariant = Split(PIDString, ",")
ReDim PID(UBound(PIDVariant))

```

```

For i = 0 To (UBound(PIDVariant) - 1)
    PID(i) = PIDVariant(i)
Next i

```

```

Set SelObj = Part.Extension.GetObjectByPersistReference((PID))

```

```

EntityType = Right(PIDVariant(UBound(PIDVariant)), 1)

```

```

If EntityType = 1 Then boolstatus = SelObj.Select2(Append, Mark)
If EntityType = 2 Then boolstatus = SelObj.Annotation.Select3(Append, Mark)
If EntityType = 3 Then boolstatus = SelObj.Select(Append, Mark)

```

```

SelectByPID = boolstatus

```

End Function

Exclude Faces Before Flattening Example (VB.NET)

This example shows how to exclude faces from a Flat-Pattern feature.

```

'-----
' Preconditions: Verify that the specified part to open exists.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Excludes 22 faces from Flat-Pattern1.
' 3. Unsuppresses Flat-Pattern1.
' 4. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere,
' do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Sub main()

        Dim Part As ModelDoc2
        Dim boolstatus As Boolean
        Dim longstatus As Long
        Dim LongWarnings As Long
        Dim swFeature As Feature
        Dim swFlatPatternFeatureData As FlatPatternFeatureData
        Dim PIDCollection As New Collection

        If SwApp Is Nothing Then Exit Sub

        Part = SwApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\FPF.SLDPRT", 1, 0, "",
longstatus, LongWarnings)
        If Part Is Nothing Then ErrorMsg(SwApp, "Failed to open FPF.SLDPRT")

        PIDCollection = PIDInitializer()

        boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.EditUnsuppress2()

        Dim View As ModelView
        View = Part.ActiveView
        Dim rect() As Double
        rect = Nothing
        View.GraphicsRedraw(rect)

        boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.EditSuppress2()
        If boolstatus = False Then ErrorMsg(SwApp, "EditSuppress Failed")

        Part = SwApp.ActiveDoc
        swFeature = Part.FirstFeature
        While Not swFeature Is Nothing
            If swFeature.GetTypeName = "FlatPattern" Then

                'Select excluded faces
                swFlatPatternFeatureData = swFeature.GetDefinition

                swFlatPatternFeatureData.AccessSelections(Part, Nothing)
                'Pick by PID
                boolstatus = SelectByPID(SwApp, Part, "selection1", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection2", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection3", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection4", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection5", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection6", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection7", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection8", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection9", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection10", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection11", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection12", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection13", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection14", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection15", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection16", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection17", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection18", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection19", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection20", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection21", PIDCollection, 1, True)
                boolstatus = SelectByPID(SwApp, Part, "selection22", PIDCollection, 1, True)
            End If
        End While
    End Sub

```

```

Dim faces(0 To 21) As Face2
Dim ii As Integer
For ii = 0 To 21

    faces(ii) = Part.SelectionManager.GetSelectedObject6(ii + 1, 1)

Next ii

Dim facesVar As Object
facesVar = faces

swFlatPatternFeatureData.ExcludedFaces = facesVar
boolstatus = swFeature.ModifyDefinition(swFlatPatternFeatureData, Part, Nothing)
If boolstatus = False Then ErrorMsg(swApp, "Selecting excluded faces failed")

boolstatus = Part.Extension.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = Part.EditUnsuppress2()
If boolstatus = False Then ErrorMsg(swApp, "EditUnsuppress Failed")

View.GraphicsRedraw(rect)

End If
swFeature = swFeature.GetNextFeature
End While

Part.ForceRebuild3(False)

End Sub

Function ErrorMsg(ByVal SwApp As SldWorks, ByVal Message As String) As String
    SwApp.SendMsgToUser2(Message, 0, 0)
    SwApp.RecordLine("**** WARNING - General")
    SwApp.RecordLine("**** " & Message)
    SwApp.RecordLine("")
    Return ""
End Function

Function PIDInitializer() As Collection

    Dim PIDCollection As New Collection
    Dim selection1 As String
    Dim selection2 As String
    Dim selection3 As String
    Dim selection4 As String
    Dim selection5 As String
    Dim selection6 As String
    Dim selection7 As String
    Dim selection8 As String
    Dim selection9 As String
    Dim selection10 As String
    Dim selection11 As String
    Dim selection12 As String
    Dim selection13 As String
    Dim selection14 As String
    Dim selection15 As String
    Dim selection16 As String
    Dim selection17 As String
    Dim selection18 As String
    Dim selection19 As String
    Dim selection20 As String
    Dim selection21 As String
    Dim selection22 As String

    'Constants
    selection1 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,103,1,0,0,120,1,117,81,77,75,2,81,20,61,38,154,17,45,50,201,69,
    selection1 = selection1 &
"6,107,203,193,165,35,32,19,229,54,88,142,152,156,65,145,15,214,70,150,211,49,205,224,215,184,36,253,148,180,194,63,18,188,212,82,34
    selection1 = selection1 & ",Type=1"

    selection2 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,79,1,0,0,120,1,109,81,93,75,2,81,16,61,38,126,68,244,144,69,246
    selection2 = selection2 &
"5,46,128,220,162,184,198,10,196,100,7,37,14,220,137,61,245,176,177,41,97,94,146,49,99,181,176,142,100,83,222,64,190,226,188,237,106
    selection2 = selection2 & ",Type=1"

    selection3 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,37,1,0,0,120,1,101,80,203,74,195,80,16,61,53,168,21,113,225,3,
    selection3 = selection3 &
"78,229,112,86,201,195,149,205,37,71,197,250,194,114,170,97,114,161,147,224,207,169,164,245,136,45,123,191,227,12,255,73,52,133,92,2
    selection3 = selection3 & ",Type=1"

    selection4 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,78,1,0,0,120,1,109,81,193,74,2,81,20,61,38,90,17,45,178,200,246
    selection4 = selection4 &
",219,34,27,217,57,144,155,111,173,176,212,105,122,80,20,63,162,96,98,143,227,154,191,110,151,140,31,147,122,64,9,241,254,175,132,96
    selection4 = selection4 & ",Type=1"

    selection5 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,95,1,0,0,120,1,109,82,77,75,2,81,20,61,102,90,17,45,50,201,69,2
    selection5 = selection5 &
"8,215,92,171,46,187,74,78,129,212,116,123,133,109,18,147,29,228,100,31,129,55,52,7,97,217,93,95,151,208,143,73,203,82,66,118,255,91
    selection5 = selection5 & ",Type=1"

    selection6 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,97,1,0,0,120,1,133,80,203,74,195,80,16,61,181,84,43,226,194,42,

```

```
selection6 = selection6 &
"235,129,218,213,119,212,45,79,219,190,252,115,151,66,82,228,100,64,139,185,217,141,21,143,178,243,9,239,205,248,111,174,61,114,213,
selection6 = selection6 & ",Type=1"

selection7 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,137,2,0,0,111,1,0,0,120,1,125,82,77,75,195,64,16,125,181,180,85,196,131,
selection7 = selection7 &
"8,77,139,189,182,60,107,129,7,171,137,192,158,11,235,215,157,100,162,109,92,144,215,93,241,142,229,253,4,222,68,31,135,117,55,60,11
selection7 = selection7 & ",Type=1"

selection8 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,97,1,0,0,120,1,101,81,77,75,2,81,20,61,38,126,68,180,200,36,23,
selection8 = selection8 &
"219,202,16,147,115,32,53,207,174,57,25,162,204,32,199,7,183,167,142,249,184,49,41,81,94,83,17,48,91,84,121,170,170,202,183,102,106,
selection8 = selection8 & ",Type=1"

selection9 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,77,1,0,0,120,1,109,81,77,75,2,81,20,61,54,168,69,180,200,34,215,
selection9 = selection9 &
"146,133,25,115,32,63,223,92,229,68,73,118,80,226,131,219,179,64,221,175,109,74,152,23,100,76,217,45,172,67,217,84,48,144,175,56,51,
selection9 = selection9 & ",Type=1"

selection10 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,101,1,0,0,120,1,125,82,203,74,195,64,20,61,181,214,7,34,98,21,2,
selection10 = selection10 &
"6,163,195,118,67,207,122,164,103,13,92,189,203,243,122,160,254,236,37,55,221,96,103,64,141,177,105,190,150,122,223,58,119,79,239,26
selection10 = selection10 & ",Type=1"

selection11 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,37,1,0,0,120,1,93,80,203,74,195,80,16,61,53,90,21,113,225,3,25,
selection11 = selection11 &
"2,229,218,15,215,36,39,55,144,83,13,147,115,157,4,127,78,37,165,135,44,217,101,123,125,247,255,35,150,169,255,218,55,254,120,101,23
selection11 = selection11 & ",Type=1"

selection12 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,77,1,0,0,120,1,109,81,209,74,2,81,16,61,102,90,17,61,100,145,18
selection12 = selection12 &
"5,0,185,100,123,197,137,146,236,160,40,251,8,253,169,61,137,107,222,250,186,164,252,132,101,135,148,144,221,255,149,88,124,22,37,16
selection12 = selection12 & ",Type=1"

selection13 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,80,2,0,0,105,1,0,0,120,1,101,82,205,74,2,81,24,61,102,90,17,45,50,201,65
selection13 = selection13 &
"114,14,164,230,187,43,110,155,156,220,65,142,31,172,15,108,183,105,89,147,127,215,37,242,83,202,178,180,224,1,54,44,228,29,196,34,6
selection13 = selection13 & ",Type=1"

selection14 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,80,1,0,0,120,1,101,81,209,74,2,65,20,61,102,90,17,61,100,145,18
selection14 = selection14 &
",114,241,230,50,38,74,50,131,2,31,220,158,26,53,92,153,148,48,47,200,8,89,45,172,67,153,148,25,200,87,156,53,204,234,84,133,91,38,1
selection14 = selection14 & ",Type=1"

selection15 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,38,1,0,0,120,1,93,80,203,74,195,64,20,61,53,214,7,226,194,7,25,
selection15 = selection15 &
"40,213,165,84,225,159,179,180,203,13,28,51,117,159,169,77,185,85,242,206,196,155,22,171,36,254,99,252,212,47,249,1,100,81,101,46,0,
selection15 = selection15 & ",Type=1"

selection16 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,78,1,0,0,120,1,109,81,93,75,2,65,20,61,182,104,69,244,144,69,24
selection16 = selection16 &
",153,1,217,217,214,42,182,152,52,103,144,231,135,91,83,163,134,107,147,18,230,37,171,98,86,203,164,142,100,82,102,32,171,56,175,155
selection16 = selection16 & ",Type=1"

selection17 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,98,1,0,0,120,1,125,81,203,74,195,64,20,61,181,214,7,226,194,42,
selection17 = selection17 &
",148,93,15,100,87,223,145,183,60,109,248,209,159,119,201,197,27,156,12,168,179,54,237,23,164,42,178,49,223,155,249,223,92,123,228,1
selection17 = selection17 & ",Type=1"

selection18 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,137,2,0,0,109,1,0,0,120,1,133,81,77,75,2,81,20,61,38,90,17,45,50,201,166
selection18 = selection18 &
",46,185,166,133,110,75,222,154,231,195,170,202,183,231,202,90,155,73,38,220,194,57,121,157,21,239,72,230,227,123,99,125,20,212,220,
selection18 = selection18 & ",Type=1"

selection19 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,79,1,0,0,120,1,101,81,205,74,2,81,20,254,76,212,34,90,100,145,2
selection19 = selection19 &
"06,128,220,108,125,25,19,38,153,65,137,31,238,141,60,211,118,221,240,207,184,4,126,64,216,46,41,248,128,127,20,133,31,138,34,41,58,
selection19 = selection19 & ",Type=1"

selection20 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,89,2,0,0,96,1,0,0,120,1,133,80,205,74,2,81,20,254,76,52,35,90,100,161,65
selection20 = selection20 &
"45,64,237,234,217,234,150,167,109,47,250,115,151,92,92,224,100,64,139,181,233,141,21,14,89,145,141,121,111,230,127,99,237,19,171,46
selection20 = selection20 & ",Type=1"

selection21 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,191,1,0,0,38,1,0,0,120,1,93,80,203,74,195,64,20,61,53,216,86,196,133,15,
selection21 = selection21 &
"87,191,92,155,156,190,129,62,213,52,189,144,105,248,231,84,186,244,136,37,123,108,39,212,246,127,68,183,17,106,252,1,7,155,101,58,6
selection21 = selection21 & ",Type=1"

selection22 =
"35,29,213,113,218,129,72,162,168,88,152,178,27,137,239,153,33,2,0,0,80,1,0,0,120,1,93,81,93,75,2,81,16,61,38,126,68,244,144,69,246,
```

```

        selection22 = selection22 &
", 61, 3, 50, 179, 252, 10, 203, 17, 147, 29, 20, 249, 224, 238, 200, 53, 45, 199, 9, 54, 214, 37, 244, 99, 210, 14, 40, 33, 187, 223, 148, 16, 76, 36, 10, 148, 104, 71, 7
        selection22 = selection22 & ", Type=1"

'Store constants in a collection
PIDCollection.Add(selection1, "selection1")
PIDCollection.Add(selection2, "selection2")
PIDCollection.Add(selection3, "selection3")
PIDCollection.Add(selection4, "selection4")
PIDCollection.Add(selection5, "selection5")
PIDCollection.Add(selection6, "selection6")
PIDCollection.Add(selection7, "selection7")
PIDCollection.Add(selection8, "selection8")
PIDCollection.Add(selection9, "selection9")
PIDCollection.Add(selection10, "selection10")
PIDCollection.Add(selection11, "selection11")
PIDCollection.Add(selection12, "selection12")
PIDCollection.Add(selection13, "selection13")
PIDCollection.Add(selection14, "selection14")
PIDCollection.Add(selection15, "selection15")
PIDCollection.Add(selection16, "selection16")
PIDCollection.Add(selection17, "selection17")
PIDCollection.Add(selection18, "selection18")
PIDCollection.Add(selection19, "selection19")
PIDCollection.Add(selection20, "selection20")
PIDCollection.Add(selection21, "selection21")
PIDCollection.Add(selection22, "selection22")

'Pass this back
PIDInitializer = PIDCollection

End Function

Function SelectByPID(ByVal SwApp As SldWorks, ByVal Part As ModelDoc2, ByVal PIDName As String, ByVal PIDCollection As
Collection, Optional ByVal Mark As Long = 0, Optional ByVal Append As Boolean = False) As Boolean

    Dim SelObj As Object
    Dim PID() As Byte
    Dim PIDVariant As Object
    Dim PIDString As String
    Dim EntityType As Long
    Dim i As Integer
    Dim boolstatus As Boolean

    'Get the string from the collection
    PIDString = ""
    PIDString = PIDCollection.Item(PIDName)

    'Parse the string into an array
    PIDVariant = Split(PIDString, ",")
    ReDim PID(UBound(PIDVariant))

    'Change to a byte array
    For i = 0 To (UBound(PIDVariant) - 1)
        PID(i) = PIDVariant(i)
    Next i

    SelObj = Part.Extension.GetObjectByPersistReference((PID))

    'Get the entity type
    EntityType = Right(PIDVariant(UBound(PIDVariant)), 1)

    If EntityType = 1 Then boolstatus = SelObj.Select2(Append, Mark)
    If EntityType = 2 Then boolstatus = SelObj.Annotation.Select3(Append, Mark)
    If EntityType = 3 Then boolstatus = SelObj.Select(Append, Mark)

    SelectByPID = boolstatus
End Function

Public swApp As SldWorks

End Class

```

Expand or Collapse FeatureManager Design Tree Nodes Example (C#)

This example shows how to traverse, expand, and collapse the nodes of a FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Open a part or assembly document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Expands all of the FeatureManager design tree nodes.
// 2. Click OK to collapse all nodes.
// 3. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;
namespace ExpandCollapseFMDesignTree_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        int traverseLevel;
        bool expandThis;

        public void Main()
        {
            int i = 0;

            ModelDoc2 myModel = default(ModelDoc2);
            FeatureManager featureMgr = default(FeatureManager);
            TreeControlItem rootNode = default(TreeControlItem);

            myModel = (ModelDoc2)swApp.ActiveDoc;
            featureMgr = myModel.FeatureManager;
            rootNode = featureMgr.GetFeatureTreeRootItem2((int)swFeatMgrPane_e.swFeatMgrPaneBottom);

            expandThis = true;

            for (i = 0; i <= 1; i++)
            {
                if ((rootNode != null))
                {
                    Debug.Print("");
                    traverseLevel = 0;
                    traverse_node(rootNode);
                }

                expandThis = false;

                if (i == 0)
                {
                    MessageBox.Show("OK to collapse all nodes?");
                }
            }
        }

        private void traverse_node(TreeControlItem node)
        {
            TreeControlItem childNode = default(TreeControlItem);
            Feature featureNode = default(Feature);
            Component2 componentNode = default(Component2);
            int nodeObjectType = 0;
            object nodeObject = null;
            string restOfString = "";
            string indent = "";
            int i = 0;
            bool displayNodeInfo = false;
            string compName = null;
            int suppr = 0;
            string supprString = "";
            int vis = 0;
            string visString = "";
            string notfloatingString = "";
            bool notfloating = false;
            object componentDoc = null;
            string docString = "";
            string refConfigName = "";

            displayNodeInfo = false;
            nodeObjectType = node.ObjectType;
            nodeObject = node.Object;
        }
    }
}
```

```

switch (nodeObjectType)
{
    case (int)swTreeControlItemType_e.swFeatureManagerItem_Feature:
        displayNodeInfo = true;
        if ((nodeObject != null))
        {
            featureNode = (Feature)nodeObject;
            restOfString = "[FEATURE: " + featureNode.Name + "]";
        }
        else
        {
            restOfString = "[FEATURE: object Null?!]";
        }
        break;
    case (int)swTreeControlItemType_e.swFeatureManagerItem_Component:
        displayNodeInfo = true;
        if ((nodeObject != null))
        {
            componentNode = (Component2)nodeObject;
            compName = componentNode.Name2;

            if ((String.IsNullOrEmpty(compName)))
            {
                compName = "???";
            }

            suppr = componentNode.GetSuppression();

            switch ((suppr))
            {
                case (int)swComponentSuppressionState_e.swComponentFullyResolved:
                    supprString = "Resolved";
                    break;
                case (int)swComponentSuppressionState_e.swComponentLightweight:
                    supprString = "Lightweight";
                    break;
                case (int)swComponentSuppressionState_e.swComponentSuppressed:
                    supprString = "Suppressed";
                    break;
            }

            vis = componentNode.Visible;

            switch ((vis))
            {
                case (int)swComponentVisibilityState_e.swComponentHidden:
                    visString = "Hidden";
                    break;
                case (int)swComponentVisibilityState_e.swComponentVisible:
                    visString = "Visible";
                    break;
            }

            notfloating = componentNode.IsFixed();

            if (notfloating == false)
            {
                notfloatingString = "Floating";
            }
            else
            {
                notfloatingString = "Fixed";
            }

            componentDoc = componentNode.GetModelDoc2();

            if (componentDoc == null)
            {
                docString = "NotLoaded";
            }
            else
            {
                docString = "Loaded";
            }

            refConfigName = componentNode.ReferencedConfiguration;

            if ((String.IsNullOrEmpty(refConfigName)))
            {
                refConfigName = "???";
            }

            restOfString = "[COMPONENT: " + compName + " " + docString + " " + supprString + " " + visString + " " +
refConfigName + "]";

```

```

    }
    else
    {
        restOfString = "[COMPONENT: object Null?!]";
    }

    break;
default:
    displayNodeInfo = true;

    if ((nodeObject != null))
    {
        restOfString = "[object type not handled]";
    }
    else
    {
        restOfString = "[object Null?!]";
    }

    break;
}

for (i = 1; i <= traverseLevel; i++)
{
    indent = indent + "  ";
}

if ((displayNodeInfo))
{
    Debug.Print(indent + node.Text + " : " + restOfString);
}

// Expand the node
node.Expanded = expandThis;
traverseLevel = traverseLevel + 1;
childNode = node.GetFirstChild();

while ((childNode != null))
{
    Debug.Print(indent + "Node is expanded: " + childNode.Expanded);
    traverse_node(childNode);
    childNode = childNode.GetNext();
}

traverseLevel = traverseLevel - 1;
}

public SldWorks swApp;
}
}

```